

Airline Industry Data Analysis

Factors Influencing Coach Ticket Prices

Course	STAT-2205 — Programming with R and Python
Name	Fahim Islam
Student ID	12310048
Dataset	flight_data.csv (129,780 observations)
Sample Size	10,000 rows (random_state / seed = 12310048)
Language	Python & R
Submitted to	Dr. Md. Siddikur Rahman
Academic Year	2023–2024

Contents

1	Introduction	3
1.1	Dataset Variables	3
1.2	Data Loading & Sampling	3
2	Q1 — Coach Ticket Price Distribution	5
2.1	Key Statistics	5
2.2	Code	5
2.3	Visualisations	7
2.4	Interpretation	8
3	Q2 — Coach Prices for 8-Hour Flights	9
3.1	Code	9
3.2	Visualisations	11
3.3	Interpretation	12
4	Q3 — Flight Delay Distribution	13
4.1	Code	13
4.2	Visualisations	15
4.3	Interpretation	16
5	Q4 — Correlation: Coach Price vs. Predictors	17
5.1	Code	17
5.2	Visualisations	19
5.3	Interpretation	19
6	Q5 — Coach vs. First-Class Price Relationship	20
6.1	Code	20
6.2	Visualisations	22
6.3	Interpretation	22
7	Q6 — In-Flight Features and Coach Price	23
7.1	Code	23
7.2	Visualisations	25
7.3	Interpretation	25
8	Q7 — Passengers vs. Flight Duration	26

8.1	Code	26
8.2	Visualisations	28
8.3	Interpretation	28
9	Q8 — Weekend vs. Weekday Pricing	29
9.1	Code	29
9.2	Visualisations	31
9.3	Interpretation	31
10	Q9 — Coach Prices: Redeye vs. Non-Redeye by Day	32
10.1	Code	32
10.2	Visualisations	34
10.3	Interpretation	34
11	Q10 — Comprehensive Statistical Analysis	36
11.1	Q10a — Summary Statistics	36
11.2	Q10b — Visualisations	36
11.3	Q10c — Hypothesis Testing	40
11.4	Q10d — Independent t-Tests	41
11.5	Q10e — Price Differences: Weekend vs. Weekday	43
11.6	Q10f — Correlation Matrix	43
11.7	Q10g — Linear Regression: Predict Coach Price	44
11.8	Q10h — Logistic Regression: Predict Redeye	47
12	Conclusions	50

1. Introduction

This report presents a comprehensive statistical analysis of **129,780** airline flight records. The objective is to reveal the key factors influencing coach ticket prices. A reproducible sample of **10,000** observations is drawn using Student ID 12310048 as the random seed.

1.1 Dataset Variables

Variable	Type	Description
miles	Numeric	Distance flown (miles)
passengers	Numeric	Number of passengers on board
delay	Numeric	Takeoff delay (minutes)
inflight_meal	Categorical	Meal included? (Yes / No)
inflight_entertainment	Categorical	Free entertainment? (Yes / No)
inflight_wifi	Categorical	Complimentary WiFi? (Yes / No)
day_of_week	Categorical	Day of the week
redeye	Categorical	Overnight flight? (Yes / No)
weekend	Categorical	Weekend flight? (Yes / No)
coach_price	Numeric	Average coach ticket price (\$)
firstclass_price	Numeric	Average first-class price (\$)
hours	Numeric	Flight duration (hours)

Table 1.1: Dataset variables, types, and descriptions

1.2 Data Loading & Sampling

```
1 import pandas as pd
2
3 df_full = pd.read_csv("flight_data.csv")
4 df = df_full.sample(n=10_000, random_state=12310048).reset_index(drop=
    True)
```

Listing 1.1: Python — Load dataset and draw 10,000-row reproducible sample

```
1 df_full <- read.csv("flight_data.csv", stringsAsFactors = FALSE)
2 names(df_full)[1] <- "miles"
3
4 set.seed(12310048)
5 idx <- sample(nrow(df_full), 10000)
6 df <- df_full[idx, ]
```

Listing 1.2: R — Load dataset and draw 10,000-row reproducible sample

2. Q1 — Coach Ticket Price Distribution

What do coach ticket prices look like? What are the high and low values? What would be considered the average? Does \$500 seem like a good price for a coach ticket?

2.1 Key Statistics

Statistic	Value
Minimum	\$84.50
Maximum	\$565.21
Mean	\$376.30
Median	\$380.50
Standard Deviation	\$66.65
25th Percentile (Q1)	\$332.18
75th Percentile (Q3)	\$425.40
Skewness	−0.3416
Kurtosis	−0.0865
Flights above \$500	1.7%

Table 2.1: Coach price descriptive statistics ($n = 10,000$)

2.2 Code

```
1 cp = df["coach_price"]
2
3 # Summary statistics
4 print(cp.describe())
5 print(f"Skewness : {cp.skew():.4f}")
6 print(f"Kurtosis : {cp.kurtosis():.4f}")
7 pct = (cp > 500).mean() * 100
8 print(f"Above $500 : {pct:.1f}%")
9
10 # Histogram + Boxplot + KDE + ECDF (2x2 grid)
11 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
12
```

```

13 axes[0,0].hist(cp, bins=50, color="steelblue", edgecolor="white",
    alpha=0.85)
14 axes[0,0].axvline(cp.mean(), color="red", linestyle="--", label=f
    "Mean ${cp.mean():.0f}")
15 axes[0,0].axvline(cp.median(), color="orange", linestyle="--", label=f
    "Median ${cp.median():.0f}")
16 axes[0,0].axvline(500, color="green", linestyle="--", label="$
    500 mark")
17 axes[0,0].set_title("Q1 - Histogram of Coach Prices")
18 axes[0,0].legend()
19
20 axes[0,1].boxplot(cp, vert=True, patch_artist=True,
    boxprops=dict(facecolor="steelblue"),
21     medianprops=dict(color="orange", linewidth=2))
22 axes[0,1].axhline(500, color="green", linestyle="--", label="$500 mark
    ")
23 axes[0,1].set_title("Q1 - Boxplot of Coach Prices")
24 axes[0,1].legend()
25
26
27 cp.plot.kde(ax=axes[1,0], color="steelblue", linewidth=2)
28 axes[1,0].set_title("Q1 - KDE Density of Coach Prices")
29
30 sorted_cp = np.sort(cp)
31 ecdf_y = np.arange(1, len(sorted_cp)+1) / len(sorted_cp)
32 axes[1,1].plot(sorted_cp, ecdf_y, color="steelblue", linewidth=2)
33 axes[1,1].axvline(500, color="green", linestyle="--", label=f"$500")
34 axes[1,1].set_title("Q1 - ECDF of Coach Prices")
35 axes[1,1].legend()
36
37 plt.tight_layout(); plt.savefig("q1_coach_price.png", dpi=150)

```

Listing 2.1: Python — Summary statistics and visualisation for coach price

```

1 cp <- df$coach_price
2
3 # Summary statistics
4 summary(cp)
5 sd(cp)
6 library(moments)
7 skewness(cp); kurtosis(cp)
8 mean(cp > 500) * 100 # % above $500
9
10 # Visualisations
11 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
12
13 hist(cp, breaks=50, col="steelblue", border="white",
14     main="Q1 - Histogram of Coach Prices", xlab="Coach Price ($)")
15 abline(v=mean(cp), col="red", lty=2, lwd=2)
16 abline(v=median(cp), col="orange", lty=2, lwd=2)
17 abline(v=500, col="green", lty=2, lwd=2)
18 legend("topleft", legend=c("Mean", "Median", "$500"),

```

```

19     col=c("red","orange","green"), lty=2, cex=0.8)
20
21 boxplot(cp, col="steelblue", main="Q1 - Boxplot of Coach Prices",
22         ylab="Coach Price ($)")
23 abline(h=500, col="green", lty=2, lwd=2)
24
25 plot(density(cp), col="steelblue", lwd=2,
26      main="Q1 - KDE of Coach Prices", xlab="Coach Price ($)")
27
28 sorted_cp <- sort(cp)
29 ecdf_y     <- seq_along(sorted_cp) / length(sorted_cp)
30 plot(sorted_cp, ecdf_y, type="l", col="steelblue", lwd=2,
31      main="Q1 - ECDF of Coach Prices",
32      xlab="Coach Price ($)", ylab="Cumulative Proportion")
33 abline(v=500, col="green", lty=2, lwd=2)
34 par(mfrow = c(1, 1))

```

Listing 2.2: R — Summary statistics and visualisation for coach price

2.3 Visualisations

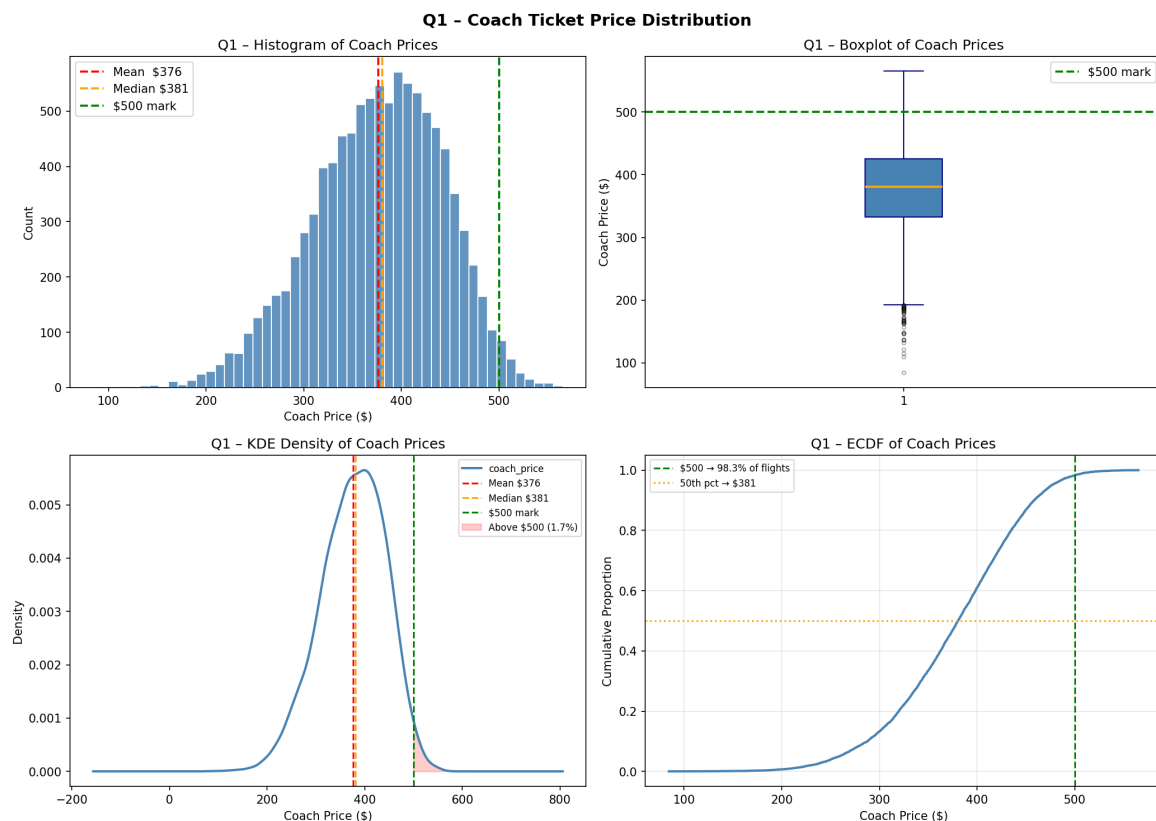


Figure 2.1: Q1 — Coach Price: Histogram, Boxplot, KDE, and ECDF

2.4 Interpretation

Interpretation

The distribution is approximately symmetric (mean $\approx$ median $\approx$ \$376–\$381). The IQR spans \$332–\$425. Only 1.7% of tickets exceed \$500, placing it in the top 2% of all fares.

Conclusion: \$500 is an expensive coach ticket, well above the market average. A typical traveller would expect to pay \$332–\$425 for a coach seat.

3. Q2 — Coach Prices for 8-Hour Flights

What are the high, low, and average prices for 8-hour-long flights? Does a \$500 ticket seem more reasonable than before?

Statistic	All Flights	8-Hour Flights
Count	10,000	182
Min Price	\$84.50	\$222.98
Max Price	\$565.21	\$556.38
Mean Price	\$376.30	\$432.18
Median	\$380.50	\$423.93
Std Dev	\$66.65	\$58.87
Above \$500	1.7%	14.8%

Table 3.1: Coach price: all flights vs. 8-hour flights

3.1 Code

```
1 df8 = df[df["hours"] == 8]["coach_price"]
2 print(df8.describe())
3 pct = (df8 > 500).mean() * 100
4 print(f"Above $500 : {pct:.1f}%")
5
6 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
7
8 # Histogram: All vs 8-hr
9 axes[0,0].hist(cp, bins=40, color="steelblue", alpha=0.6, label="All
    flights")
10 axes[0,0].hist(df8, bins=20, color="darkorange", alpha=0.8, label="8-
    hour flights")
11 axes[0,0].axvline(500, color="green", linestyle="--", label="$500 mark
    ")
12 axes[0,0].set_title("Q2 - All vs 8-Hour Flights"); axes[0,0].legend()
13
14 # Side-by-side boxplot
15 bp = axes[0,1].boxplot([cp, df8], labels=["All Flights", "8-Hour
    Flights"],
16                             patch_artist=True)
17 for patch, c in zip(bp["boxes"], ["steelblue", "darkorange"]):
```

```

18     patch.set_facecolor(c); patch.set_alpha(0.75)
19     axes[0,1].axhline(500, color="green", linestyle="--"); axes[0,1].
        set_ylabel("Coach Price ($)")
20
21     # Violin by hour
22     hours_order = sorted(df["hours"].unique())
23     prices_by_hour = [df[df["hours"]==h]["coach_price"].values for h in
        hours_order]
24     axes[1,0].violinplot(prices_by_hour, positions=hours_order,
        showmedians=True)
25     axes[1,0].set_title("Q2 - Price by Duration (Violin)")
26
27     # Bar: mean price per hour
28     mean_ph = df.groupby("hours")["coach_price"].mean()
29     axes[1,1].bar(mean_ph.index, mean_ph.values, color="steelblue",
        edgecolor="white")
30     axes[1,1].axhline(cp.mean(), color="red", linestyle="--", label="
        Overall mean")
31     axes[1,1].axhline(500, color="green", linestyle="--", label="
        $500")
32     axes[1,1].legend(); axes[1,1].set_title("Q2 - Mean Price by Hour")
33     plt.tight_layout(); plt.savefig("q2_8hour_prices.png", dpi=150)

```

Listing 3.1: Python — Filter 8-hour flights and compare price distributions

```

1 df8 <- df[df$hours == 8, "coach_price"]
2 summary(df8)
3 mean(df8 > 500) * 100      # % above $500
4
5 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
6
7 # Histogram: all vs 8-hr
8 hist(cp, breaks=40, col=rgb(0.27,0.51,0.71,0.6),
9     main="Q2 - All vs 8-Hour", xlab="Coach Price ($)")
10 hist(df8, breaks=20, col=rgb(1,0.55,0,0.8), add=TRUE)
11 legend("topleft", c("All", "8-hour"), fill=c("steelblue", "darkorange"))
12
13 # Side-by-side boxplot
14 boxplot(cp, df8, names=c("All Flights", "8-Hour"), col=c("steelblue", "
    darkorange"),
15     main="Q2 - Comparison Boxplot", ylab="Coach Price ($)")
16 abline(h=500, col="green", lty=2, lwd=2)
17
18 # Bar: mean price per hour
19 avg_ph <- tapply(df$coach_price, df$hours, mean)
20 barplot(avg_ph, col="steelblue", xlab="Hours", ylab="Avg Coach Price (
    $)",
21     main="Q2 - Mean Coach Price by Duration")
22 abline(h=mean(cp), col="red", lty=2); abline(h=500, col="green", lty
    =2)
23

```

```

24 # Violin via vioplot
25 if (!require("vioplot")) install.packages("vioplot")
26 library(vioplot)
27 vioplot(cp, df8, names=c("All", "8-hr"), col=c("steelblue", "darkorange"),
28       ),
29       main="Q2 - Violin: All vs 8-Hour")
29 abline(h=500, col="green", lty=2)
30 par(mfrow = c(1, 1))

```

Listing 3.2: R — Filter 8-hour flights and compare price distributions

3.2 Visualisations

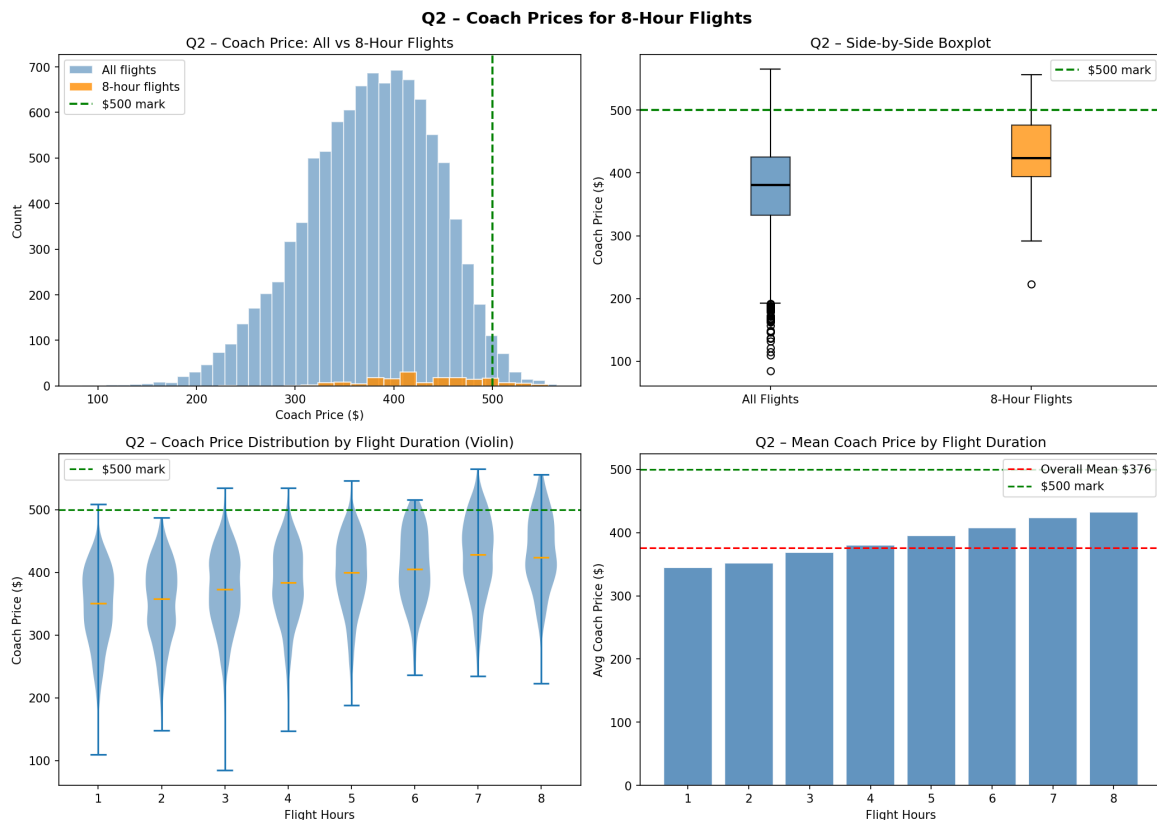


Figure 3.1: Q2 — 8-Hour Flight Prices: Histogram, Boxplot, Violin, and Mean-by-Hour

3.3 Interpretation

Interpretation

8-hour flights average **\$432.18**, roughly \$56 above the overall mean. **14.8%** of 8-hour tickets exceed \$500.

Conclusion: \$500 is more justifiable for long-haul travel but remains above average even for the longest flights.

4. Q3 — Flight Delay Distribution

How are flight delay times distributed? How often are there significant delays? What kinds of delays are typical?

Metric	Value
Mean Delay	13.61 min
Median Delay	10.00 min
Standard Deviation	47.76 min
Maximum Delay	1,524 min
No Delay (0 min)	7.6%
Flights delayed > 15 min	17.1% (connection risk)
Flights delayed > 60 min	0.1% (severe)

Table 4.1: Flight delay distribution statistics

4.1 Code

```
1 delay = df["delay"]
2
3 print(f"Mean    : {delay.mean():.2f} min")
4 print(f"Median  : {delay.median():.2f} min")
5 print(f"Std     : {delay.std():.2f} min")
6 print(f"Max     : {delay.max():.0f} min")
7 print(f"> 15 min : {(delay>15).mean()*100:.1f}% (connection risk)")
8 print(f"> 60 min : {(delay>60).mean()*100:.1f}% (severe)")
9
10 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
11
12 # Histogram (capped at 120 min)
13 delay_cap = delay[delay <= 120]
14 axes[0,0].hist(delay_cap, bins=50, color="tomato", edgecolor="white")
15 axes[0,0].axvline(15, color="orange", linestyle="--", label="15 min (connection)")
16 axes[0,0].axvline(60, color="darkred", linestyle="--", label="60 min (severe)")
17 axes[0,0].set_title("Q3 - Delay Histogram (<=120 min)"); axes[0,0].legend()
18
```

```

19 # Boxplot
20 axes[0,1].boxplot(delay, patch_artist=True, boxprops=dict(facecolor="
    tomato"),
21                   medianprops=dict(color="darkred", linewidth=2))
22 axes[0,1].set_title("Q3 - Boxplot of Delay Times")
23
24 # ECDF
25 sorted_d = np.sort(delay); ecdf_d = np.arange(1,len(sorted_d)+1)/len(
    sorted_d)
26 axes[1,0].plot(sorted_d, ecdf_d, color="tomato", linewidth=2)
27 axes[1,0].axvline(15, color="orange", linestyle="--", label="15 min")
28 axes[1,0].axvline(60, color="darkred", linestyle="--", label="60 min")
29 axes[1,0].set_xlim(0,200); axes[1,0].set_title("Q3 - ECDF"); axes
    [1,0].legend()
30
31 # Delay-category bar
32 cats = {"No delay":( delay==0).sum(), "1-15 min":((delay>0)&(delay
    <=15)).sum(),
33         "16-60 min":((delay>15)&(delay<=60)).sum(), ">60 min":(delay
    >60).sum())}
34 axes[1,1].bar(cats.keys(), cats.values(), color=["seagreen","steelblue
    ", "orange", "darkred"])
35 axes[1,1].set_title("Q3 - Flights by Delay Category")
36 plt.tight_layout(); plt.savefig("q3_delays.png", dpi=150)

```

Listing 4.1: Python — Delay distribution, thresholds, and visualisations

```

1 delay <- df$delay
2
3 mean(delay); median(delay); sd(delay); max(delay)
4 mean(delay > 15) * 100      # connection risk
5 mean(delay > 60) * 100      # severe
6
7 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
8
9 # Histogram
10 hist(delay[delay <= 120], breaks=50, col="tomato", border="white",
11       main="Q3 - Delay Histogram (<=120 min)", xlab="Delay (minutes)")
12 abline(v=15, col="orange", lty=2, lwd=2)
13 abline(v=60, col="darkred", lty=2, lwd=2)
14
15 # Boxplot
16 boxplot(delay, col="tomato", main="Q3 - Boxplot of Delays", ylab="
    Delay (min)")
17
18 # ECDF
19 plot(ecdf(delay), xlim=c(0,200), col="tomato", lwd=2,
20       main="Q3 - ECDF of Delays", xlab="Delay (min)")
21 abline(v=c(15,60), col=c("orange","darkred"), lty=2, lwd=2)
22
23 # Category bar

```

```

24 cats <- c("No delay" = sum(delay==0),
25           "1-15 min" = sum(delay>0 & delay<=15),
26           "16-60 min" = sum(delay>15 & delay<=60),
27           ">60 min" = sum(delay>60))
28 barplot(cats, col=c("seagreen","steelblue","orange","darkred"),
29         main="Q3 - Flights by Delay Category", ylab="Count")
30 par(mfrow = c(1, 1))

```

Listing 4.2: R — Delay distribution and visualisations

4.2 Visualisations

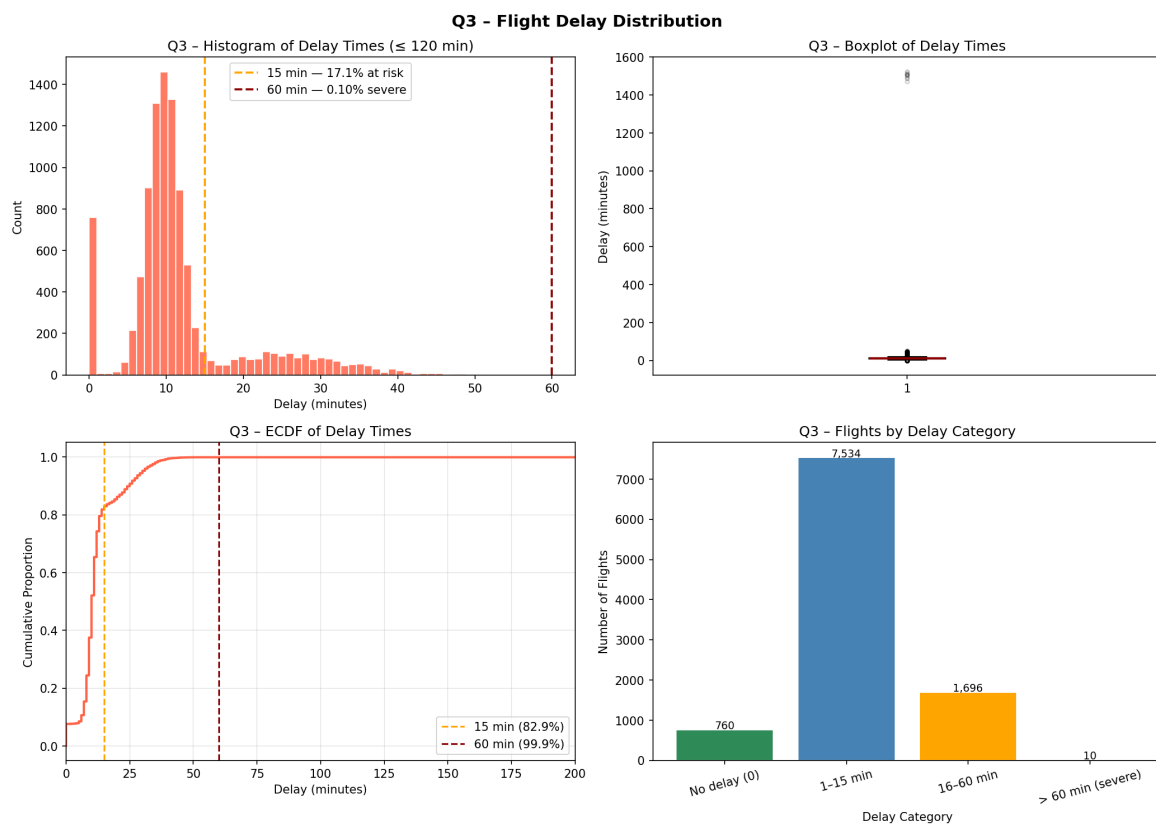


Figure 4.1: Q3 — Flight Delay Distribution: Histogram, Boxplot, ECDF, and Category Bar

4.3 Interpretation

Interpretation

The delay distribution is strongly right-skewed. Most flights experience little to no delay (median = 10 min). However, **17.1%** of flights exceed the 15-minute connection-risk threshold. Severe delays exceeding 60 minutes affect only **0.1%** of flights but can have extreme magnitudes (max = 1,524 min).

Conclusion: Typical delays are small and manageable; connection windows under 15 minutes carry meaningful risk.

5. Q4 — Correlation: Coach Price vs. Predictors

How does the coach price correlate with flight distance, passengers, delay, and flight duration?

Predictor	Pearson r	Direction	p-value	Significant?
miles	0.3454	Positive	< 0.001	Yes
passengers	0.1529	Positive	< 0.001	Yes
delay	-0.0181	Negative	0.0707	No
hours	0.3425	Positive	< 0.001	Yes

Table 5.1: Pearson correlations between coach price and numeric predictors

5.1 Code

```
1 from scipy import stats
2 import numpy as np
3
4 numeric_vars = ["miles", "passengers", "delay", "hours"]
5 for v in numeric_vars:
6     r, p = stats.pearsonr(df[v], df["coach_price"])
7     print(f" {v:<12}: r={r:+.4f}  p={p:.4f}  {'sig' if p<0.05 else 'n'
8           .s.'}")
9
10 fig, axes = plt.subplots(2, 3, figsize=(18, 10))
11 colors = ["steelblue", "darkorange", "tomato", "seagreen"]
12
13 for ax, v, c in zip(axes.flat[:4], numeric_vars, colors):
14     ax.scatter(df[v], df["coach_price"], alpha=0.1, s=5, color=c)
15     slope, intercept = np.polyfit(df[v], df["coach_price"], 1)
16     x = np.linspace(df[v].min(), df[v].max(), 100)
17     ax.plot(x, slope*x + intercept, color="black", linewidth=2)
18     r, _ = stats.pearsonr(df[v], df["coach_price"])
19     ax.set_title(f"Coach Price vs {v.capitalize()} (r={r:+.3f})")
20
21 # Correlation bar chart
22 r_vals = [stats.pearsonr(df[v], df["coach_price"])[0] for v in
23           numeric_vars]
24 axes[1,1].barh(numeric_vars, r_vals,
25               color=["steelblue" if r>=0 else "tomato" for r in
26                     r_vals])
```

```

24 axes[1,1].set_title("Pearson r with Coach Price")
25
26 # Miles vs Hours coloured by coach price
27 sc = axes[1,2].scatter(df["miles"], df["hours"], c=df["coach_price"],
28                       cmap="coolwarm", alpha=0.3, s=5)
29 plt.colorbar(sc, ax=axes[1,2], label="Coach Price ($)")
30 plt.tight_layout(); plt.savefig("q4_correlations.png", dpi=150)

```

Listing 5.1: Python — Pearson correlation tests and scatter plots

```

1 for (v in c("miles","passengers","delay","hours")) {
2   ct <- cor.test(df[[v]], df$coach_price)
3   cat(v, round(ct$estimate,4), round(ct$p.value,4), "\n")
4 }
5
6 par(mfrow = c(2, 3), mar = c(4, 4, 3, 1))
7 colors <- c("steelblue","darkorange","tomato","seagreen")
8 for (i in seq_along(c("miles","passengers","delay","hours"))){
9   v <- c("miles","passengers","delay","hours")[i]
10  plot(df[[v]], df$coach_price, pch=16, cex=0.3, col=colors[i],
11       xlab=v, ylab="Coach Price ($) ",
12       main=paste0("Coach Price vs ", v))
13  abline(lm(coach_price ~ df[[v]], data=df), col="black", lwd=2)
14 }
15
16 # Correlation bar
17 r_vals <- sapply(c("miles","passengers","delay","hours"),
18                function(v) cor(df[[v]], df$coach_price))
19 barplot(r_vals, col=ifelse(r_vals>=0,"steelblue","tomato"),
20        main="Pearson r with Coach Price", ylab="Pearson r")
21 par(mfrow = c(1, 1))

```

Listing 5.2: R — Pearson correlation tests and scatter plots

5.2 Visualisations

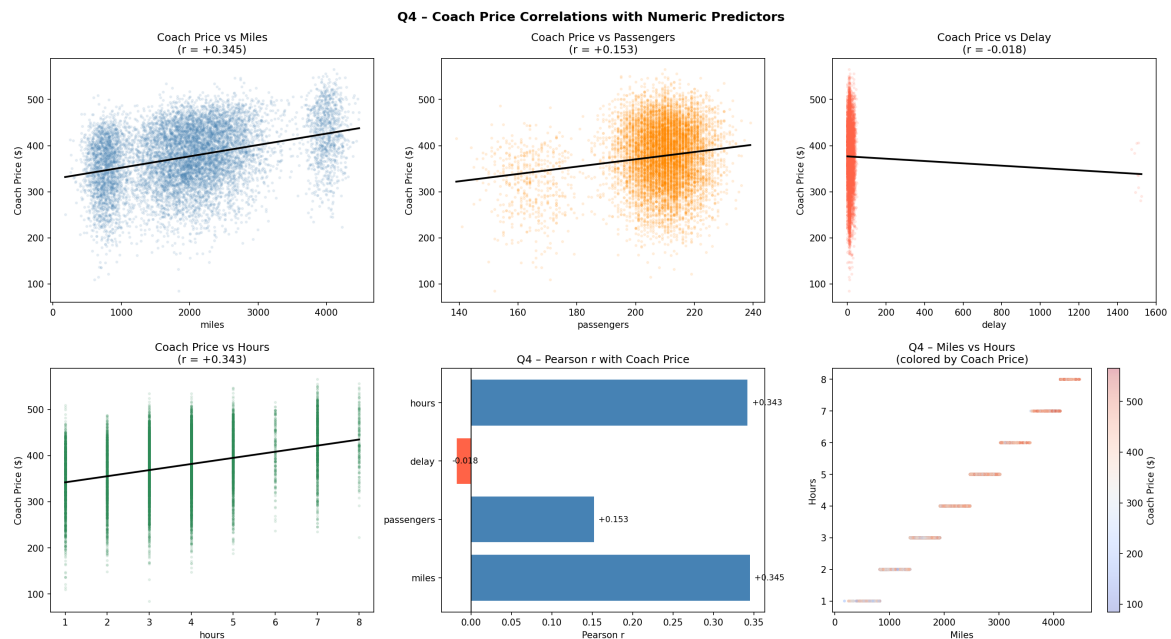


Figure 5.1: Q4 — Scatter plots, correlation bar chart, and miles vs. hours coloured by coach price

5.3 Interpretation

Interpretation

Miles ($r = 0.345$) and **hours** ($r = 0.343$) are the strongest positive predictors of coach price. **Passengers** show a weaker but significant positive correlation ($r = 0.153$). **Delay** is not a statistically significant pricing factor ($p = 0.071$).

Conclusion: Route distance and flight duration are the primary drivers of coach price among continuous predictors.

6. Q5 — Coach vs. First-Class Price Relationship

What is the relationship between coach and first-class prices? Do flights with higher coach prices always have higher first-class prices?

Metric	Value
Pearson r	0.7555
Average FC / Coach ratio	3.93×
Average coach price	\$376.30
Average first-class price	\$1,451.91
p-value	< 0.001

Table 6.1: Coach vs. first-class relationship metrics

6.1 Code

```
1 r5, p5 = stats.pearsonr(df["coach_price"], df["firstclass_price"])
2 ratio = (df["firstclass_price"] / df["coach_price"]).mean()
3 print(f"r={r5:.4f}  p={p5:.2e}  Ratio: {ratio:.2f}x")
4
5 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
6
7 # Scatter + regression
8 slope, intercept = np.polyfit(df["coach_price"], df["firstclass_price"], 1)
9 axes[0,0].scatter(df["coach_price"], df["firstclass_price"],
10                  alpha=0.12, s=5, color="purple")
11 x = np.linspace(df["coach_price"].min(), df["coach_price"].max(), 200)
12 axes[0,0].plot(x, slope*x+intercept, color="black", linewidth=2)
13 axes[0,0].set_title(f"Q5 - Coach vs First-Class (r={r5:.3f})")
14
15 # Hexbin
16 hb = axes[0,1].hexbin(df["coach_price"], df["firstclass_price"],
17                       gridsize=40, cmap="Purples", mincnt=1)
18 plt.colorbar(hb, ax=axes[0,1])
19 axes[0,1].set_title("Q5 - Hexbin Density")
20
21 # Ratio histogram
22 price_ratio = df["firstclass_price"] / df["coach_price"]
```

```

23 axes[1,0].hist(price_ratio, bins=40, color="mediumpurple", edgecolor="
    white")
24 axes[1,0].axvline(ratio, color="red", linestyle="--", label=f"Mean {
    ratio:.2f}x")
25 axes[1,0].set_title("Q5 - FC/Coach Price Ratio"); axes[1,0].legend()
26
27 # Average price bar
28 bars = axes[1,1].bar(["Coach", "First-Class"],
29                      [df["coach_price"].mean(), df["firstclass_price"
30                      ].mean()],
31                      color=["steelblue", "mediumpurple"], width=0.5)
32 for bar, val in zip(bars, [df["coach_price"].mean(), df["
33     firstclass_price"].mean()]):
34     axes[1,1].text(bar.get_x()+bar.get_width()/2, val+20,
35                     f"${val:,.0f}", ha="center", fontweight="bold")
36 axes[1,1].set_title("Q5 - Average Prices")
37 plt.tight_layout(); plt.savefig("q5_coach_vs_first.png", dpi=150)

```

Listing 6.1: Python — Correlation test and coach vs. first-class plots

```

1 ct5 <- cor.test(df$coach_price, df$firstclass_price)
2 ratio <- mean(df$firstclass_price / df$coach_price)
3 cat("r =", round(ct5$estimate,4), " p =", ct5$p.value)
4 cat("Ratio:", round(ratio,2), "x")
5
6 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
7
8 plot(df$coach_price, df$firstclass_price, pch=16, cex=0.3, col="purple
    ",
9      xlab="Coach Price ($)", ylab="First-Class Price ($)",
10     main=paste0("Q5 - Coach vs First-Class (r=", round(ct5$estimate,3)
11     ,")"))
12
13 abline(lm(firstclass_price ~ coach_price, data=df), col="black", lwd
14     =2)
15
16 price_ratio <- df$firstclass_price / df$coach_price
17 hist(price_ratio, breaks=40, col="mediumpurple", border="white",
18     main="Q5 - FC/Coach Ratio", xlab="Price Ratio")
19 abline(v=ratio, col="red", lty=2, lwd=2)
20
21 barplot(c(mean(df$coach_price), mean(df$firstclass_price)),
22     names.arg=c("Coach", "First-Class"),
23     col=c("steelblue", "mediumpurple"),
24     main="Q5 - Average Prices", ylab="Avg Price ($)")
25
26 boxplot(df$coach_price, df$firstclass_price,
27     names=c("Coach", "First-Class"), col=c("steelblue", "
28     mediumpurple"),
29     main="Q5 - Price Comparison Boxplot", ylab="Price ($)")
30 par(mfrow = c(1, 1))

```

Listing 6.2: R — Correlation test and coach vs. first-class plots

6.2 Visualisations

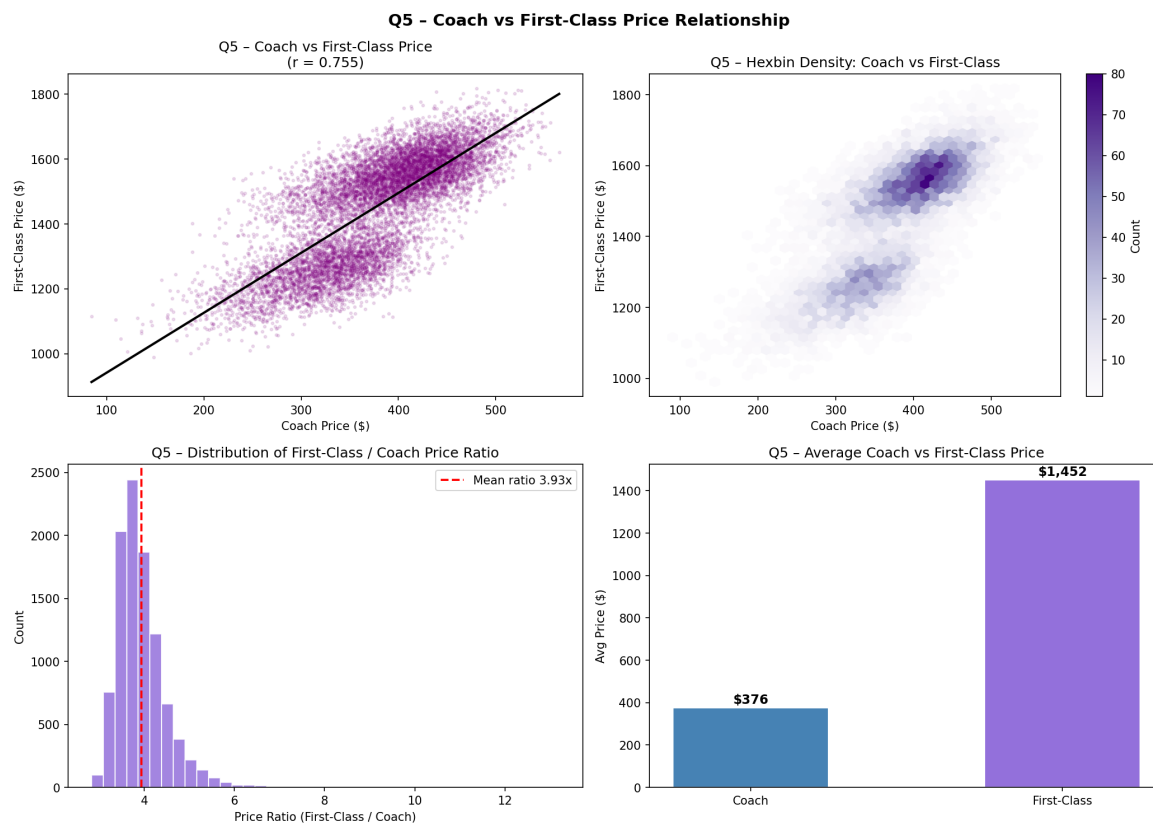


Figure 6.1: Q5 — Coach vs. First-Class: Scatter, Hexbin, Ratio Histogram, and Average Bar

6.3 Interpretation

Interpretation

There is a **strong positive** and statistically significant relationship between coach and first-class prices ($r = 0.756$, $p < 0.001$). Higher coach prices generally predict higher first-class prices. The relationship is not perfectly linear, indicating other factors also influence first-class pricing. On average, first-class tickets cost **3.93×** more than coach tickets.

7. Q6 — In-Flight Features and Coach Price

What is the relationship between coach prices and in-flight features? Which features are associated with the highest increase in price?

Feature	With Feature	Without	Premium	p-value
Inflight Meal	\$389.75	\$370.51	+\$19.24	< 0.001
Entertainment	\$389.34	\$322.30	+\$67.04	< 0.001
WiFi	\$382.81	\$314.79	+\$68.02	< 0.001

Table 7.1: Coach price premiums by in-flight feature

7.1 Code

```
1 feats = ["inflight_meal", "inflight_entertainment", "inflight_wifi"]
2 feat_results = {}
3 for f in feats:
4     yes = df[df[f]=="Yes"]["coach_price"]
5     no = df[df[f]=="No"] ["coach_price"]
6     t, p = stats.ttest_ind(yes, no)
7     feat_results[f] = {"yes":yes.mean(),"no":no.mean(),"diff":yes.mean()
8                        -no.mean(),"p":p}
9     print(f"{f}: Yes=${yes.mean():.2f} No=${no.mean():.2f} +${yes.
10            mean()-no.mean():.2f} p={p:.4f}")
11
12 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
13 # Grouped boxplots
14 # ... (see full code in Airline_Analysis_Enhanced.py)
15 # Price premium bar
16 short = ["Meal", "Entertainment", "WiFi"]
17 diffs = [feat_results[f]["diff"] for f in feats]
18 axes[0,1].bar(short, diffs, color=["steelblue", "darkorange", "seagreen"
19 ])
20 for i, d in enumerate(diffs):
21     axes[0,1].text(i, d+0.5, f"+${d:.2f}", ha="center", fontweight="
22                     bold")
23 axes[0,1].set_title("Q6 - Price Premium per Feature")
24 # Grouped bar: Yes vs No
```



```

24 x = np.arange(len(short)); w = 0.35
25 axes[1,0].bar(x-w/2,[feat_results[f]["yes"] for f in feats],w,label="
    With",color="steelblue")
26 axes[1,0].bar(x+w/2,[feat_results[f]["no"] for f in feats],w,label="
    Without",color="lightsteelblue")
27 axes[1,0].set_xticks(x); axes[1,0].set_xticklabels(short); axes[1,0].
    legend()
28
29 # Violin for WiFi
30 vp = axes[1,1].violinplot([df[df["inflight_wifi"]=="Yes"]["coach_price
    "].values,
31                             df[df["inflight_wifi"]=="No"] ["
                                coach_price"].values],
32                             positions=[1,2], showmedians=True)
33 axes[1,1].set_xticks([1,2]); axes[1,1].set_xticklabels(["WiFi: Yes", "
    WiFi: No"])
34 plt.tight_layout(); plt.savefig("q6_inflight_features.png", dpi=150)

```

Listing 7.1: Python — Independent t-tests for each in-flight feature

```

1 feats <- c("inflight_meal","inflight_entertainment","inflight_wifi")
2 for (f in feats) {
3   g1 <- df[df[[f]]=="Yes","coach_price"]
4   g2 <- df[df[[f]]=="No", "coach_price"]
5   tt <- t.test(g1, g2)
6   cat(f, round(mean(g1)-mean(g2),2), round(tt$p.value,4), "\n")
7 }
8
9 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
10
11 # Boxplot by feature
12 boxplot(coach_price ~ inflight_meal, data=df,
13         col=c("steelblue","lightsteelblue"), main="Price by Meal")
14 boxplot(coach_price ~ inflight_entertainment, data=df,
15         col=c("darkorange","moccasin"), main="Price by
16             Entertainment")
17 boxplot(coach_price ~ inflight_wifi, data=df,
18         col=c("seagreen","lightgreen"), main="Price by WiFi")
19
20 # Premium bar
21 diffs <- c(
22   Meal = mean(df[df$inflight_meal=="Yes","coach_price"]) -
23           mean(df[df$inflight_meal=="No", "coach_price"]),
24   Entertainment = mean(df[df$inflight_entertainment=="Yes","coach_
25       price"]) -
26                   mean(df[df$inflight_entertainment=="No", "coach_
27       price"]),
28   WiFi = mean(df[df$inflight_wifi=="Yes","coach_price"]) -
29           mean(df[df$inflight_wifi=="No", "coach_price"]))
30 barplot(diffs, col=c("steelblue","darkorange","seagreen"),
31         main="Price Premium per Feature", ylab="Price Difference ($)")

```

```
29 par(mfrow = c(1, 1))
```

Listing 7.2: R — Independent t-tests for each in-flight feature

7.2 Visualisations

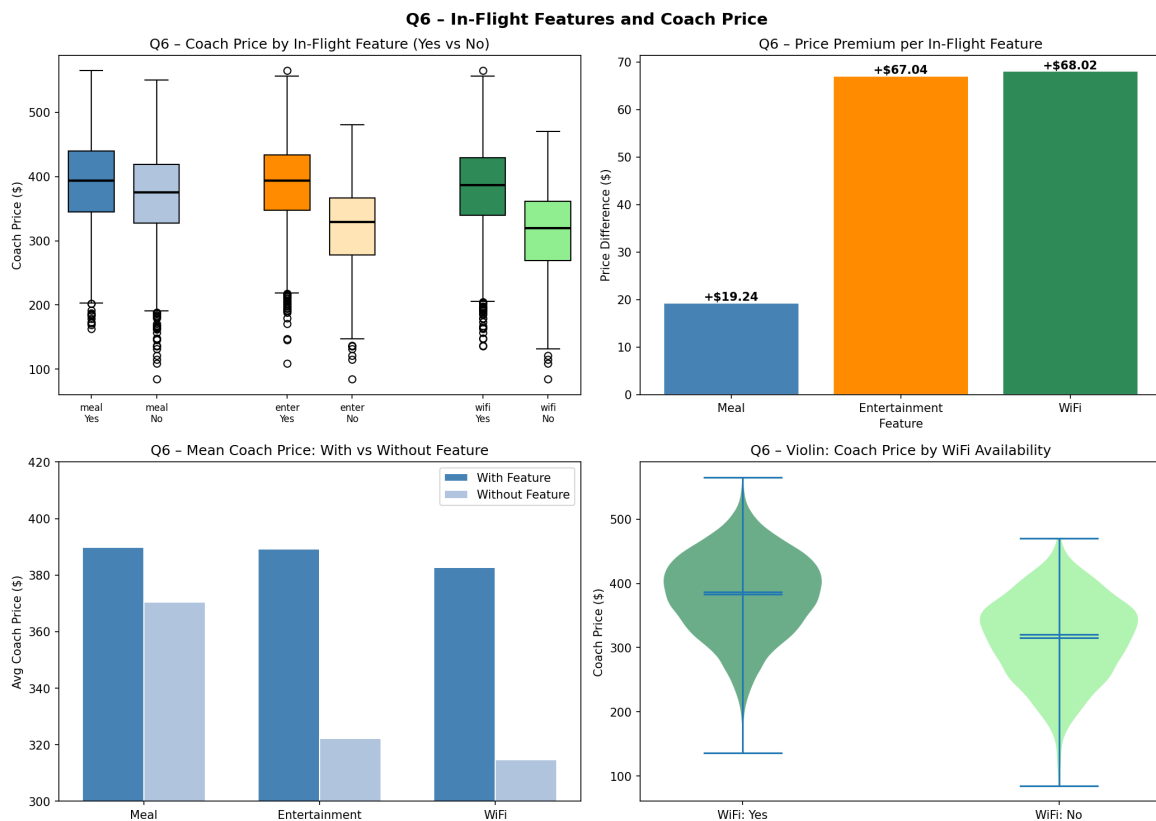


Figure 7.1: Q6 — In-Flight Features: Boxplots, Price Premium Bar, Grouped Bar, and WiFi Violin

7.3 Interpretation

Interpretation

All three in-flight services are associated with **significantly higher** coach prices. **WiFi (+\$68.02)** and **Entertainment (+\$67.04)** carry the largest premiums; Meal adds a smaller but still significant **+\$19.24**.

Conclusion: Premium cabin amenities are strongly linked to pricing strategy. WiFi and entertainment represent the highest pricing leverage for airlines.

8. Q7 — Passengers vs. Flight Duration

How does the number of passengers change in relation to the length of flights?

Hours	Avg. Passengers
1	208.2
2	207.7
3	207.6
4	207.5
5	207.1
6	208.1
7	207.9
8	207.6
Pearson r -0.011 ($p = 0.273$, not significant)	

Table 8.1: Average passengers by flight duration

8.1 Code

```
1 r7, p7 = stats.pearsonr(df["hours"], df["passengers"])
2 print(f"r={r7:.4f}  p={p7:.4f}")
3
4 avg_pax = df.groupby("hours")["passengers"].mean()
5 print(avg_pax)
6
7 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
8
9 axes[0,0].scatter(df["hours"], df["passengers"], alpha=0.1, s=5, color
10                  ="teal")
11 slope7, b7 = np.polyfit(df["hours"], df["passengers"], 1)
12 x7 = np.linspace(df["hours"].min(), df["hours"].max(), 100)
13 axes[0,0].plot(x7, slope7*x7+b7, color="black", linewidth=2, label=f"r
14                  ={r7:.3f}")
15 axes[0,0].set_title("Q7 - Passengers vs Hours"); axes[0,0].legend()
16
17 avg_pax.plot(kind="bar", ax=axes[0,1], color="teal", edgecolor="white"
18              )
19 axes[0,1].axhline(df["passengers"].mean(), color="red", linestyle="--"
20                  , label="Overall mean")
```

```

17 axes[0,1].set_title("Q7 - Avg Passengers by Hour"); axes[0,1].legend()
18
19 # Boxplot by hour
20 hours_order = sorted(df["hours"].unique())
21 pax_by_hour = [df[df["hours"]==h]["passengers"].values for h in
    hours_order]
22 bp7 = axes[1,0].boxplot(pax_by_hour, positions=hours_order,
    patch_artist=True, widths=0.6)
23 for patch in bp7["boxes"]: patch.set_facecolor("teal")
24 axes[1,0].set_title("Q7 - Passengers by Hour (Boxplot)")
25
26 axes[1,1].hist(df["passengers"], bins=40, color="teal", edgecolor="
    white")
27 axes[1,1].axvline(df["passengers"].mean(), color="red", linestyle
    = "--")
28 axes[1,1].axvline(df["passengers"].median(), color="orange", linestyle
    = "--")
29 axes[1,1].set_title("Q7 - Histogram of Passengers")
30 plt.tight_layout(); plt.savefig("q7_passengers_hours.png", dpi=150)

```

Listing 8.1: Python — Passenger vs. duration correlation and visualisations

```

1 cor.test(as.numeric(df$hours), df$passengers)
2
3 avg_pax <- tapply(df$passengers, df$hours, mean)
4 print(round(avg_pax, 1))
5
6 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
7
8 plot(df$hours, df$passengers, pch=16, cex=0.3, col="teal",
9      xlab="Flight Hours", ylab="Passengers",
10     main=paste0("Q7 - Passengers vs Hours (r=",
11                round(cor(df$hours,df$passengers),3),")"))
12 abline(lm(passengers ~ hours, data=df), col="black", lwd=2)
13
14 barplot(avg_pax, col="teal", xlab="Hours", ylab="Avg Passengers",
15         main="Q7 - Avg Passengers by Hour")
16 abline(h=mean(df$passengers), col="red", lty=2)
17
18 boxplot(passengers ~ hours, data=df, col="teal",
19         xlab="Flight Hours", ylab="Passengers",
20         main="Q7 - Passengers by Hour (Boxplot)")
21
22 hist(df$passengers, breaks=40, col="teal", border="white",
23      main="Q7 - Histogram of Passengers", xlab="Passengers")
24 par(mfrow = c(1, 1))

```

Listing 8.2: R — Passenger vs. duration correlation and visualisations

8.2 Visualisations

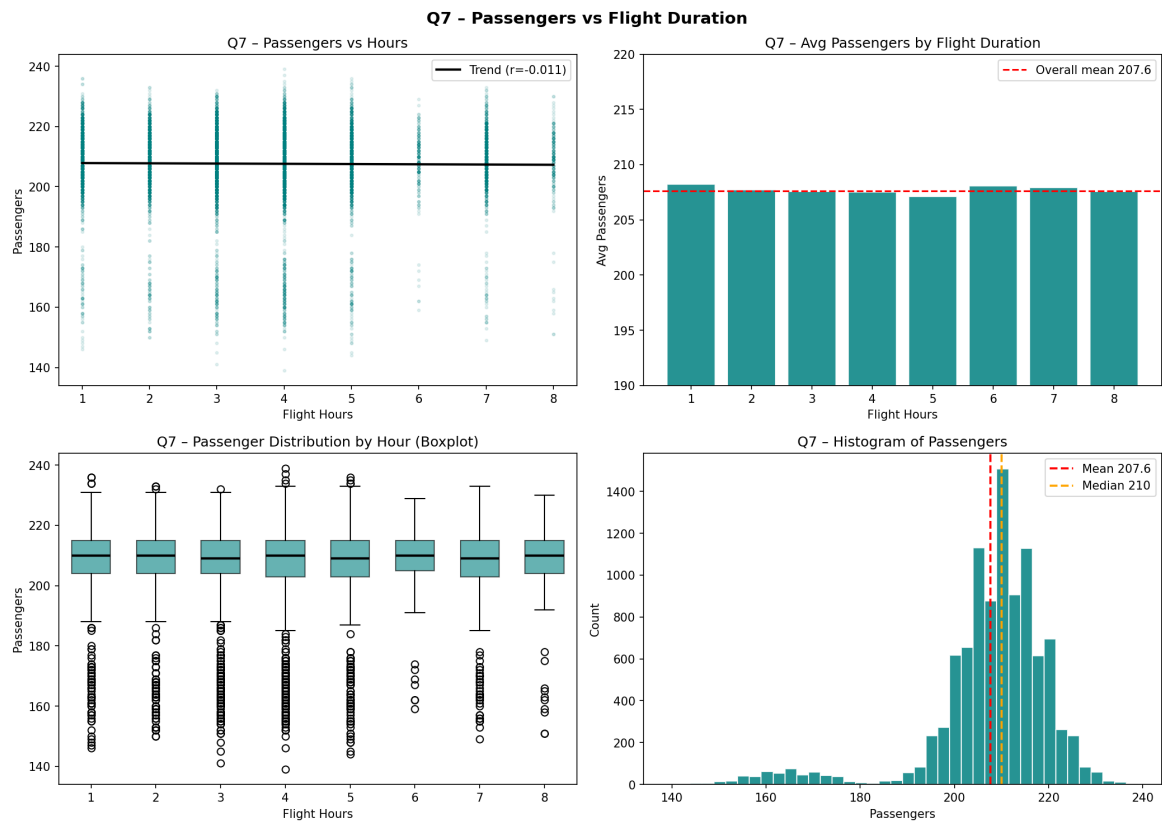


Figure 8.1: Q7 — Passengers vs. Duration: Scatter, Bar, Boxplot, and Histogram

8.3 Interpretation

Interpretation

Pearson $r = -0.011$ ($p = 0.273$) — essentially **no relationship** between flight duration and passenger count. Average passengers remain stable at **207–208** across all flight durations.

Conclusion: Airlines fill their aircraft consistently regardless of flight length. Route type does not strongly predict capacity utilisation in this dataset.

9. Q8 — Weekend vs. Weekday Pricing

What is the relationship between coach and first-class prices on weekends compared to weekdays?

Class	Weekend	Weekday	Difference	p-value
Coach	\$405.51	\$323.05	+\$82.47	< 0.001
First-Class	\$1,558.32	\$1,257.89	+\$300.43	< 0.001

Table 9.1: Price comparison: weekend vs. weekday

9.1 Code

```
1 for pc in ["coach_price", "firstclass_price"]:
2     wk = df[df["weekend"]=="Yes"][pc]
3     nwk = df[df["weekend"]=="No"][pc]
4     t, p = stats.ttest_ind(wk, nwk)
5     print(f"{pc}: Weekend=${wk.mean():.2f} Weekday=${nwk.mean():.2f}
6           "
7           f"Diff=${wk.mean()-nwk.mean():.2f} p={p:.4f}")
8
9 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
10
11 for i, (pc, col) in enumerate(zip(["coach_price", "firstclass_price"],
12                                  ["steelblue", "mediumpurple"])):
13     bp = axes[0, i].boxplot(
14         [df[df["weekend"]=="Yes"][pc].values, df[df["weekend"]=="No"][
15             pc].values],
16         labels=["Weekend", "Weekday"], patch_artist=True)
17     for patch in bp["boxes"]: patch.set_facecolor(col)
18     axes[0, i].set_title(f"Q8 - {pc.replace('_', ' ').title()}")
19
20 # Violin: coach by weekend
21 vp = axes[1, 0].violinplot(
22     [df[df["weekend"]=="Yes"]["coach_price"].values,
23      df[df["weekend"]=="No"]["coach_price"].values],
24     positions=[1, 2], showmedians=True)
25 for j, b in enumerate(vp["bodies"]): b.set_facecolor(["coral", "
26               steelblue"][j])
27 axes[1, 0].set_xticks([1, 2]); axes[1, 0].set_xticklabels(["Weekend", "
28               Weekday"])
```

```

25 axes[1,0].set_title("Q8 - Violin: Coach by Weekend")
26
27 # Bar: mean prices for both classes
28 cats = ["Coach\nWeekend", "Coach\nWeekday", "FC\nWeekend", "FC\nWeekday"
29         ]
29 means = [df[df["weekend"]=="Yes"]["coach_price"].mean(),
30          df[df["weekend"]=="No" ]["coach_price"].mean(),
31          df[df["weekend"]=="Yes"]["firstclass_price"].mean(),
32          df[df["weekend"]=="No" ]["firstclass_price"].mean()]
33 bars = axes[1,1].bar(cats, means, color=["coral", "steelblue", "
34         darkorchid", "mediumpurple"])
34 for bar, val in zip(bars, means):
35     axes[1,1].text(bar.get_x()+bar.get_width()/2, val+15, f"${val:,.0f
36         }",
37                    ha="center", fontsize=9, fontweight="bold")
37 plt.tight_layout(); plt.savefig("q8_weekend_weekday.png", dpi=150)

```

Listing 9.1: Python — Weekend vs. weekday t-tests and visualisations

```

1 for (pc in c("coach_price", "firstclass_price")) {
2   wk  <- df[df$weekend=="Yes", pc]
3   nwk <- df[df$weekend=="No",  pc]
4   tt  <- t.test(wk, nwk)
5   cat(pc, round(mean(wk)-mean(nwk),2), round(tt$p.value,4), "\n")
6 }
7
8 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
9
10 boxplot(coach_price ~ weekend, data=df, col=c("lightcoral", "steelblue"
11        ),
12         main="Q8 - Coach: Weekend vs Weekday", ylab="Coach Price ($)")
12 boxplot(firstclass_price ~ weekend, data=df, col=c("darkorchid", "
13        mediumpurple"),
14         main="Q8 - First-Class: Weekend vs Weekday", ylab="First-Class
15        Price ($)")
14
15 if (!require("vioplot")) install.packages("vioplot"); library(vioplot)
16 vioplot(df[df$weekend=="Yes", "coach_price"], df[df$weekend=="No", "
17        coach_price"],
18         names=c("Weekend", "Weekday"), col=c("coral", "steelblue"),
19         main="Q8 - Violin: Coach by Weekend")
19
20 means <- c(mean(df[df$weekend=="Yes", "coach_price"]),
21            mean(df[df$weekend=="No",  "coach_price"]),
22            mean(df[df$weekend=="Yes", "firstclass_price"]),
23            mean(df[df$weekend=="No",  "firstclass_price"]))
24 barplot(means, names.arg=c("Coach\nWknd", "Coach\nWkdy", "FC\nWknd", "FC\
25        nWkdy"),
26         col=c("coral", "steelblue", "darkorchid", "mediumpurple"),
27         main="Q8 - Mean Prices by Class & Weekend", ylab="Avg Price ($

```

```
27 par(mfrow = c(1, 1))
```

Listing 9.2: R — Weekend vs. weekday t-tests and visualisations

9.2 Visualisations

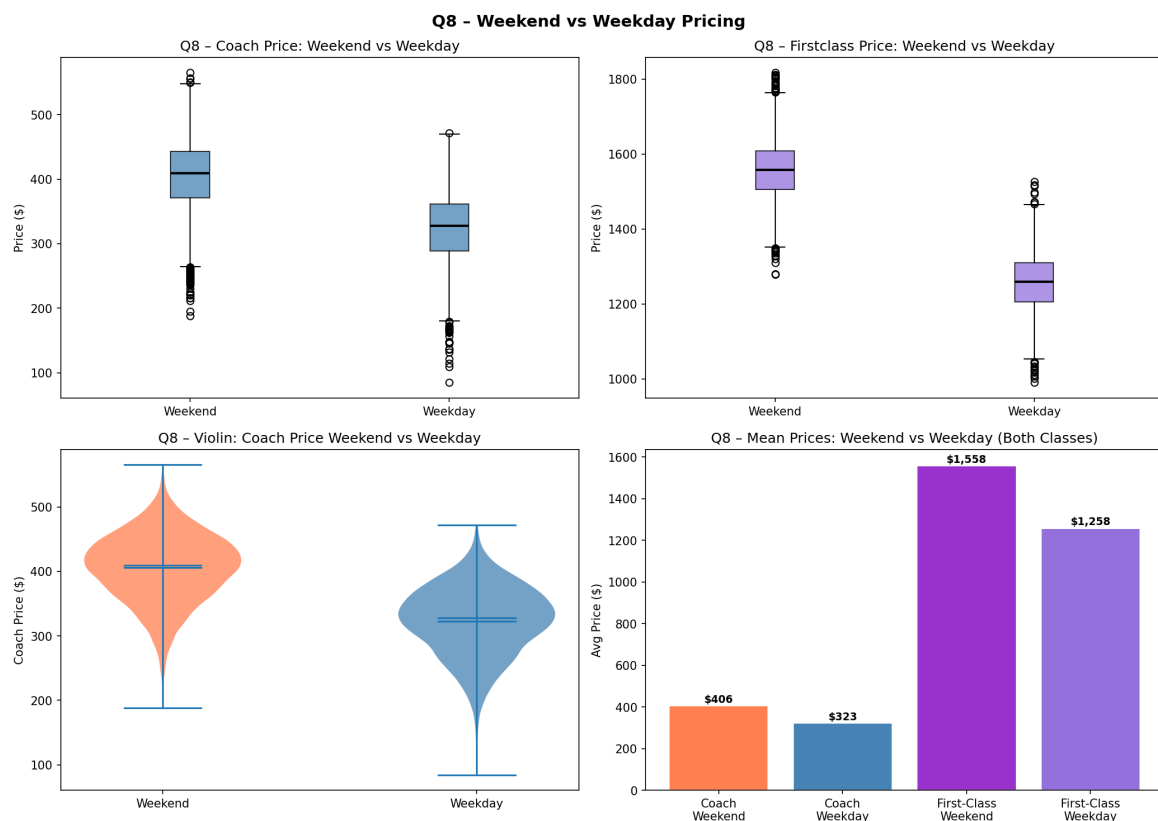


Figure 9.1: Q8 — Weekend vs. Weekday: Boxplots, Violin, and Mean Price Bar

9.3 Interpretation

Interpretation

Both coach and first-class prices are **significantly higher on weekends** ($p < 0.001$). Coach fares rise by **+\$82.47** on weekends; first-class fares by **+\$300.43**.

Conclusion: Demand-based pricing is clearly at work. Travellers seeking lower fares should fly on weekdays.

10. Q9 — Coach Prices: Redeye vs. Non-Redeye by Day

How do coach prices differ for redeyes and non-redeyes on each day of the week?

Day	Non-Redeye	Redeye	Discount
Monday	\$325.53	\$257.30	−\$68.23
Tuesday	\$328.16	\$272.33	−\$55.83
Wednesday	\$327.52	\$260.95	−\$66.57
Thursday	\$323.92	\$265.93	−\$57.99
Friday	\$405.15	\$344.37	−\$60.78
Saturday	\$414.04	\$358.88	−\$55.16
Sunday	\$404.67	\$343.07	−\$61.60

Table 10.1: Average coach price by day of week and redeye status

10.1 Code

```
1 import seaborn as sns
2
3 days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
4         "Sunday"]
5 pivot = df.pivot_table(values="coach_price", index="day_of_week",
6                          columns="redeye", aggfunc="mean").reindex(days)
7 print(pivot.round(2))
8
9
10 # Grouped bar
11 x = np.arange(len(days)); w = 0.35
12 axes[0,0].bar(x-w/2, pivot["No"].values, w, label="Non-Redeye", color
13              ="steelblue")
14 axes[0,0].bar(x+w/2, pivot["Yes"].values, w, label="Redeye", color
15              ="tomato")
16 axes[0,0].set_xticks(x); axes[0,0].set_xticklabels([d[:3] for d in
17              days])
18 axes[0,0].set_title("Q9 - Redeye vs Non-Redeye (Bar)"); axes[0,0].
19 legend()
```

```

16
17 # Line chart
18 axes[0,1].plot(days, pivot["No"].values, "o-", color="steelblue",
19               label="Non-Redeye")
19 axes[0,1].plot(days, pivot["Yes"].values, "s--", color="tomato",
20               label="Redeye")
20 axes[0,1].fill_between(range(len(days)), pivot["Yes"].values,
21                       pivot["No"].values, alpha=0.15, color="gray")
22 axes[0,1].set_xticks(range(len(days))); axes[0,1].set_xticklabels([d
23                           [:3] for d in days])
23 axes[0,1].set_title("Q9 - Price Trend (Line)"); axes[0,1].legend()
24
25 # Heatmap
26 sns.heatmap(pivot.T, annot=True, fmt=".0f", cmap="RdYlGn_r",
27             ax=axes[1,0], xticklabels=[d[:3] for d in days])
28 axes[1,0].set_title("Q9 - Heatmap: Price by Day & Redeye")
29
30 # Discount bar
31 discounts = pivot["No"].values - pivot["Yes"].values
32 axes[1,1].bar([d[:3] for d in days], discounts, color="darkred",
33              edgecolor="white")
33 axes[1,1].set_title("Q9 - Redeye Discount by Day")
34 plt.tight_layout(); plt.savefig("q9_redeye_by_day.png", dpi=150)

```

Listing 10.1: Python — Pivot table and grouped bar/line/heatmap charts

```

1 library(reshape2); library(ggplot2)
2
3 day_ord <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "
4             Saturday", "Sunday")
5
6 pivot <- tapply(df$coach_price, list(df$day_of_week, df$redeye), mean)
7 pivot <- pivot[day_ord, ]
8 print(round(pivot, 2))
9
10 # Grouped bar via ggplot2
11 melt_df <- melt(pivot)
12 colnames(melt_df) <- c("Day", "Redeye", "AvgPrice")
13 ggplot(melt_df, aes(Day, AvgPrice, fill=Redeye)) +
14   geom_bar(stat="identity", position="dodge") +
15   scale_fill_manual(values=c("No"="steelblue", "Yes"="tomato")) +
16   labs(title="Q9 - Coach Price: Redeye vs Non-Redeye", y="Avg Coach
17         Price ($)") +
18   theme_minimal() +
19   theme(axis.text.x=element_text(angle=30, hjust=1))
20
21 # Discount bar
22 discounts <- pivot[, "No"] - pivot[, "Yes"]
23 barplot(discounts, col="darkred", main="Q9 - Redeye Discount by Day",
24         ylab="Price Discount ($)", las=2)

```

```

24 # Heatmap via ggplot
25 ggplot(melt_df, aes(Day, Redeye, fill=AvgPrice)) +
26   geom_tile(color="white") +
27   geom_text(aes(label=round(AvgPrice,0)), size=3.5) +
28   scale_fill_gradient2(low="green", mid="yellow", high="red", midpoint
29     =340) +
29   labs(title="Q9 - Heatmap: Coach Price by Day & Redeye") +
30   theme_minimal() + theme(axis.text.x=element_text(angle=30, hjust=1))

```

Listing 10.2: R — Pivot table and grouped bar/heatmap for redeye by day

10.2 Visualisations

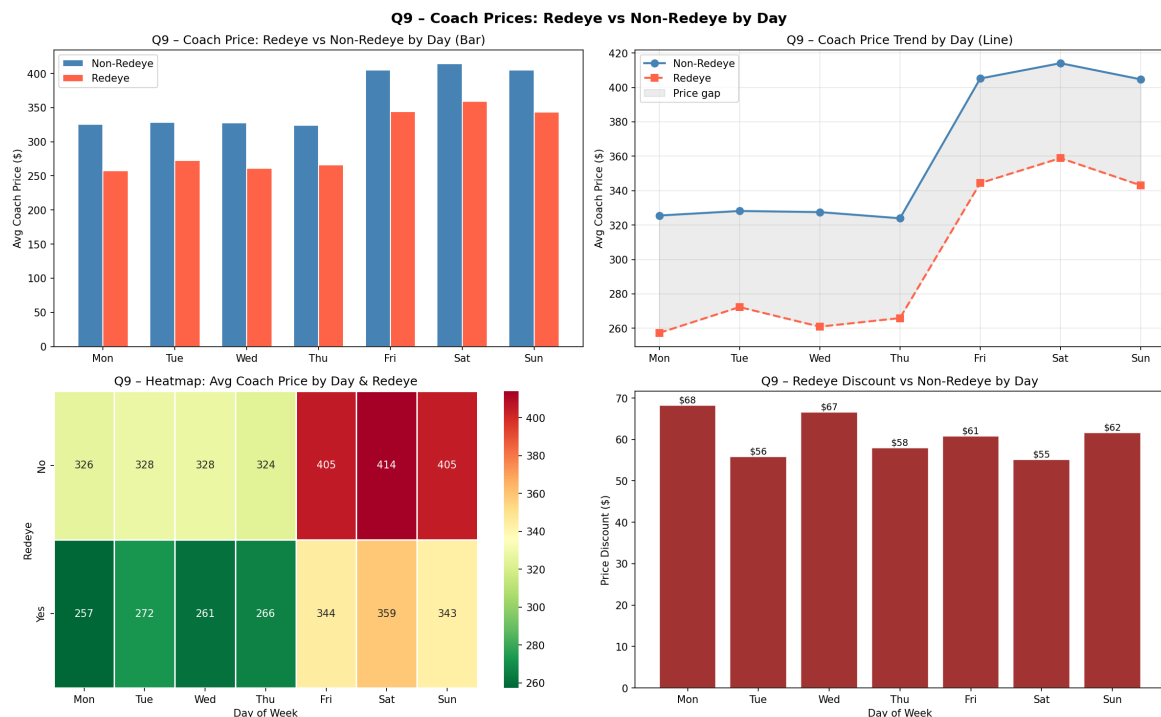


Figure 10.1: Q9 — Redeye vs. Non-Redeye: Grouped Bar, Line, Heatmap, and Discount Bar

10.3 Interpretation

Interpretation

Non-redeye flights are **consistently more expensive** than redeye flights on every day of the week, with discounts ranging from **\$55–\$68**. Weekend prices are highest for both flight types, with Saturday peaking.

Conclusion: The cheapest coach option is a weekday redeye (Monday or Wednesday).

Friday–Sunday redeye flights are more expensive than weekday non-redeye flights due to demand.

11. Q10 — Comprehensive Statistical Analysis

11.1 Q10a — Summary Statistics

Variable	Mean	Median	Std Dev	Min	Max	Skewness
miles	1983.9	1968.0	934.9	178	4,479	0.087
passengers	207.7	210.0	12.84	139	239	−0.634
delay (min)	13.61	10.00	47.76	0	1,524	19.87
coach_price (\$)	376.30	380.50	66.65	84.50	565.21	−0.342
firstclass (\$)	1451.9	1499.5	162.8	990.0	1817.5	−0.445
hours	3.53	4.00	1.72	1	8	0.202

Table 11.1: Summary statistics for all numeric variables

```
1 numeric_cols = ["miles", "passengers", "delay", "coach_price", "firstclass_price", "hours"]
2 summary = df[numeric_cols].agg(["mean", "median", "std", "min", "max",
3                                lambda x: x.skew(),
4                                lambda x: x.kurtosis()]).round(3)
5 summary.index = ["mean", "median", "std", "min", "max", "skewness", "kurtosis"]
6 print(summary.to_string())
```

Listing 11.1: Python — Comprehensive summary statistics

```
1 library(psych)
2 num_cols <- c("miles", "passengers", "delay", "coach_price", "firstclass_price", "hours")
3 describe(df[num_cols])
```

Listing 11.2: R — Comprehensive summary statistics

11.2 Q10b — Visualisations

```
1 hist_colors = ["steelblue", "darkorange", "tomato", "seagreen", "mediumpurple", "teal"]
2
```

```

3 # Histograms
4 fig, axes = plt.subplots(2, 3, figsize=(16, 8))
5 for ax, col, c in zip(axes.flat, numeric_cols, hist_colors):
6     ax.hist(df[col], bins=35, color=c, edgecolor="white", alpha=0.85)
7     ax.axvline(df[col].mean(), color="red", linestyle="--", label
8               = "Mean")
9     ax.axvline(df[col].median(), color="orange", linestyle=":", label
10              = "Median")
11     ax.set_title(f"Histogram - {col}"); ax.legend(fontsize=7)
12 plt.savefig("q10b_histograms.png", dpi=150)
13
14 # Boxplots
15 fig, axes = plt.subplots(2, 3, figsize=(16, 8))
16 for ax, col, c in zip(axes.flat, numeric_cols, hist_colors):
17     ax.boxplot(df[col], patch_artist=True, boxprops=dict(facecolor=c,
18                 alpha=0.7),
19               medianprops=dict(color="black", linewidth=2))
20     ax.set_title(f"Boxplot - {col}")
21 plt.savefig("q10b_boxplots.png", dpi=150)
22
23 # Bar charts (categorical)
24 cat_cols = ["inflight_meal", "inflight_entertainment", "inflight_wifi",
25            "redeye", "weekend", "day_of_week"]
26 fig, axes = plt.subplots(2, 3, figsize=(16, 8))
27 for ax, col in zip(axes.flat, cat_cols):
28     counts = df[col].value_counts()
29     ax.bar(counts.index, counts.values, color="darkorange", edgecolor=
30           "white")
31     ax.set_title(f"Bar Chart - {col}"); ax.tick_params(axis="x",
32               rotation=30)
33 plt.savefig("q10b_barcharts.png", dpi=150)
34
35 # KDE plots
36 fig, axes = plt.subplots(2, 3, figsize=(16, 8))
37 for ax, col, c in zip(axes.flat, numeric_cols, hist_colors):
38     df[col].plot.kde(ax=ax, color=c, linewidth=2)
39     ax.set_title(f"KDE - {col}")
40 plt.savefig("q10b_kde.png", dpi=150)

```

Listing 11.3: Python — Histograms, boxplots, bar charts, and KDE plots

```

1 num_cols <- c("miles", "passengers", "delay", "coach_price", "firstclass_
2   price", "hours")
3 colors <- c("steelblue", "darkorange", "tomato", "seagreen", "
4   mediumpurple", "teal")
5
6 # Histograms
7 par(mfrow=c(2,3), mar=c(4,4,3,1))
8 for (i in seq_along(num_cols)) {
9     hist(df[[num_cols[i]]], breaks=35, col=colors[i], border="white",
10         main=paste("Hist -", num_cols[i]), xlab=num_cols[i])

```

```

9   abline(v=mean(df[[num_cols[i]]],na.rm=T), col="red", lty=2)
10  abline(v=median(df[[num_cols[i]]],na.rm=T), col="orange", lty=3)
11 }
12
13 # Boxplots
14 par(mfrow=c(2,3), mar=c(4,4,3,1))
15 for (i in seq_along(num_cols)) {
16   boxplot(df[[num_cols[i]]], col=colors[i],
17          main=paste("Box -",num_cols[i]), ylab=num_cols[i])
18 }
19
20 # Bar charts (categorical)
21 cat_cols <- c("inflight_meal","inflight_entertainment","inflight_wifi"
22             ,
23             "redeye","weekend","day_of_week")
24 par(mfrow=c(2,3), mar=c(5,4,3,1))
25 for (col in cat_cols) {
26   barplot(table(df[[col]]), col="darkorange", main=col, las=2)
27 }
28 par(mfrow=c(1,1))

```

Listing 11.4: R — Histograms, boxplots, and bar charts

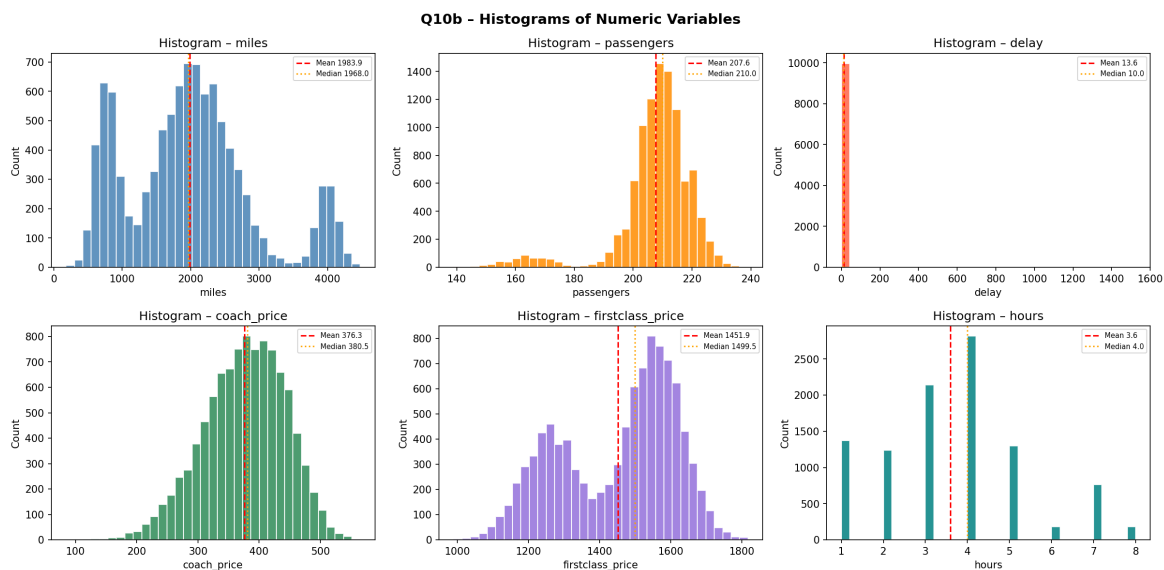


Figure 11.1: Q10b — Histograms of all numeric variables with mean and median lines

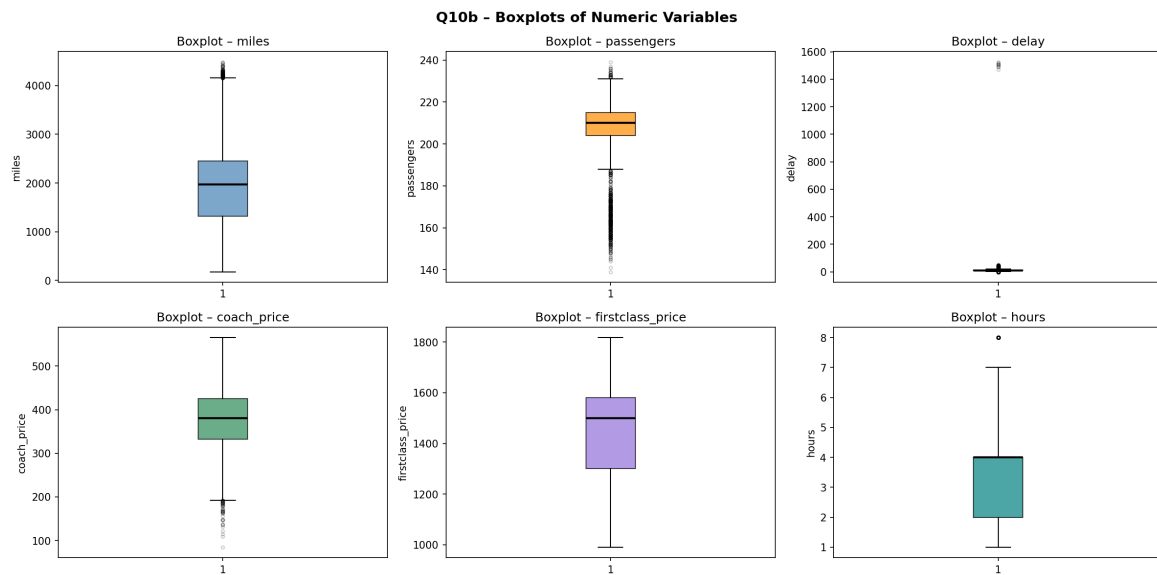


Figure 11.2: Q10b — Boxplots of all numeric variables

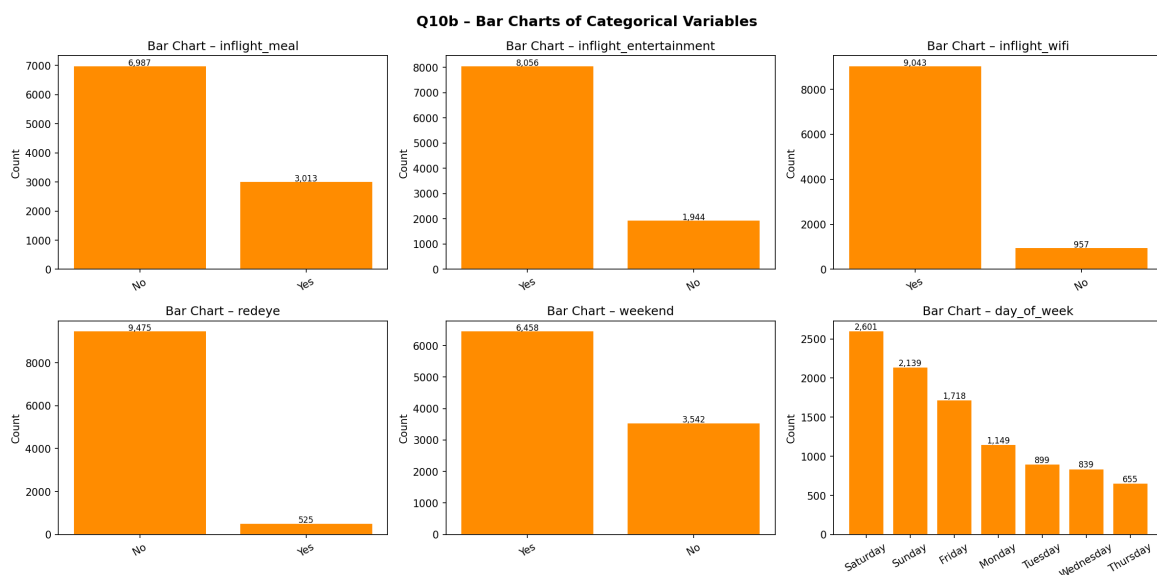


Figure 11.3: Q10b — Bar charts of all categorical variables

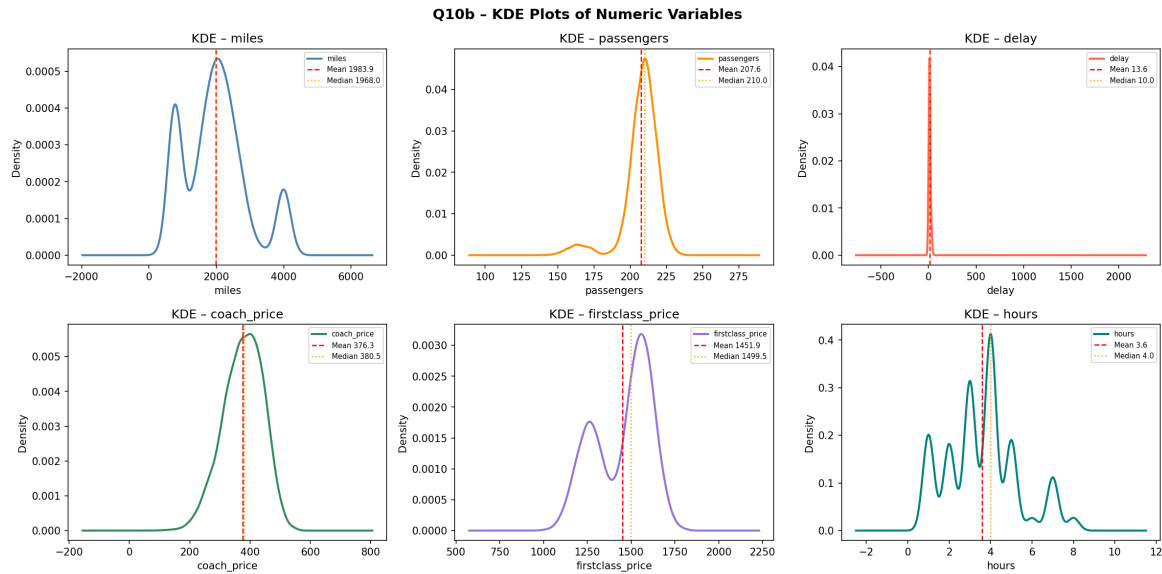


Figure 11.4: Q10b — KDE density plots of all numeric variables

11.3 Q10c — Hypothesis Testing

Test	Statistic	p-value	Decision
Z-test ($H_0: \mu = \$350$)	$z = 39.46$	< 0.001	Reject H_0
t-test ($H_0: \mu = \$380$)	$t = -5.55$	< 0.001	Reject H_0
Chi-sq: redeye $\times$ meal	$\chi^2 = 4.34$	0.0372	Reject H_0 (dependent)
Chi-sq: redeye $\times$ weekend	$\chi^2 = 0.021$	0.885	Fail to reject

Table 11.2: Hypothesis test results

```

1 from scipy.stats import norm
2 import pandas as pd
3
4 cp = df["coach_price"]
5
6 # Z-test (H0: mu = $350)
7 z = (cp.mean() - 350) / (cp.std() / np.sqrt(len(cp)))
8 p_z = 2 * (1 - norm.cdf(abs(z)))
9 print(f"Z-test: z={z:.4f} p={p_z:.4e}")
10
11 # One-sample t-test (H0: mu = $380)
12 t, p_t = stats.ttest_1samp(cp, 380)
13 print(f"t-test: t={t:.4f} p={p_t:.4e}")
14
15 # Chi-square tests
16 for a, b in [("redeye", "inflight_meal"), ("redeye", "weekend")]:
17     ct = pd.crosstab(df[a], df[b])

```

```

18 chi2, p_chi, dof, _ = stats.chi2_contingency(ct)
19 print(f"Chi-sq {a}x{b}: chi2={chi2:.4f} p={p_chi:.4f}")

```

Listing 11.5: Python — Z-test, one-sample t-test, and chi-square tests

```

1 cp <- df$coach_price
2
3 # Z-test (H0: mu = $350)
4 z <- (mean(cp)-350) / (sd(cp)/sqrt(length(cp)))
5 p_z <- 2 * pnorm(-abs(z))
6 cat("Z-test: z=", round(z,4), " p=", p_z, "\n")
7
8 # One-sample t-test (H0: mu = $380)
9 t.test(cp, mu=380)
10
11 # Chi-square tests
12 chisq.test(table(df$redeye, df$inflight_meal))
13 chisq.test(table(df$redeye, df$weekend))

```

Listing 11.6: R — Z-test, one-sample t-test, and chi-square tests

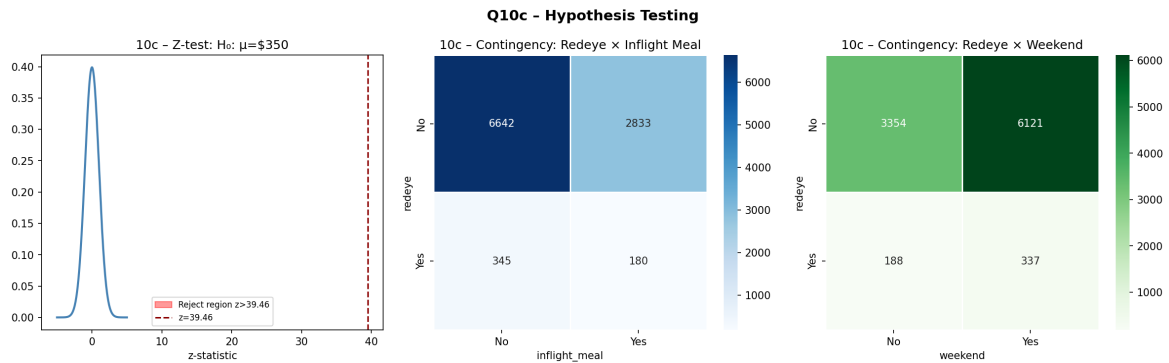


Figure 11.5: Q10c — Z-distribution with rejection region and contingency heatmaps

11.4 Q10d — Independent t-Tests

Comparison	Yes	No	t-stat	p-value
Weekend vs. Weekday	\$405.49	\$323.47	73.41	< 0.001
Redeye vs. Non-Redeye	\$319.22	\$379.47	-20.58	< 0.001
Meal vs. No Meal	\$389.64	\$370.51	13.36	< 0.001
Entertainment vs. None	\$389.34	\$322.30	35.20	< 0.001
WiFi vs. No WiFi	\$382.81	\$314.79	40.11	< 0.001

Table 11.3: Independent t-test results for coach price group comparisons

```

1 groups = [
2     ("weekend",          "coach_price", "Weekend"),
3     ("redeye",           "coach_price", "Redeye"),
4     ("inflight_meal",     "coach_price", "Meal"),
5     ("inflight_entertainment", "coach_price", "Entertainment"),
6     ("inflight_wifi",     "coach_price", "WiFi"),
7 ]
8 for col, target, label in groups:
9     g1 = df[df[col]=="Yes"][target]
10    g2 = df[df[col]=="No"][target]
11    t, p = stats.ttest_ind(g1, g2)
12    print(f"{label}: Yes=${g1.mean():.2f} No=${g2.mean():.2f} t={t:.3f} p={p:.4f}")

```

Listing 11.7: Python — Independent t-tests for all group comparisons

```

1 groups <- list(
2   c("weekend", "Weekend"), c("redeye", "Redeye"),
3   c("inflight_meal", "Meal"), c("inflight_entertainment", "Entertainment"),
4   c("inflight_wifi", "WiFi"))
5 for (g in groups) {
6   g1 <- df[df[[g[1]]]=="Yes", "coach_price"]
7   g2 <- df[df[[g[1]]]=="No", "coach_price"]
8   tt <- t.test(g1, g2)
9   cat(g[2], round(mean(g1), 2), round(mean(g2), 2),
10      round(tt$statistic, 3), round(tt$p.value, 4), "\n")
11 }

```

Listing 11.8: R — Independent t-tests for group mean comparisons

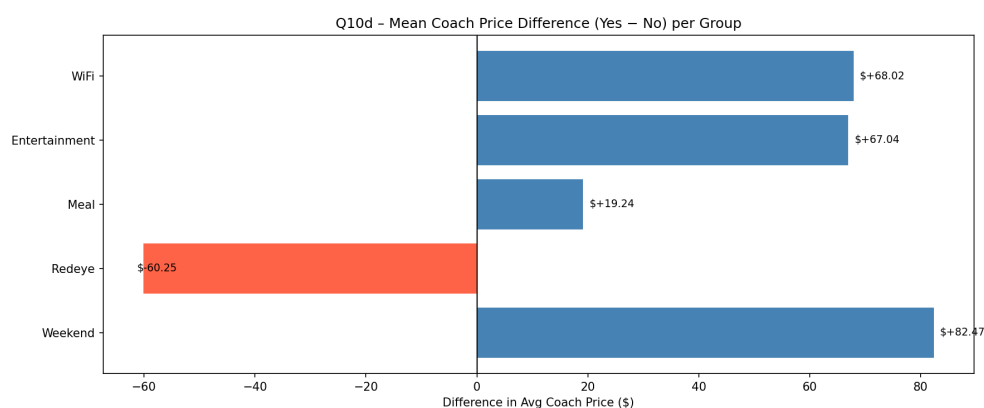


Figure 11.6: Q10d — Forest plot: mean coach price difference (Yes – No) per group

11.5 Q10e — Price Differences: Weekend vs. Weekday

Price Class	Weekend	Weekday	Difference
Coach	\$405.51	\$323.47	+\$82.47
First-Class	\$1,558.32	\$1,257.89	+\$300.43

Table 11.4: Weekend vs. weekday average price differences

11.6 Q10f — Correlation Matrix

	miles	pax	delay	coach	first	hours
miles	1.000	−0.012	−0.007	0.340	0.288	0.985
passengers	−0.012	1.000	0.005	0.160	0.007	−0.011
delay	−0.007	0.005	1.000	−0.017	−0.027	−0.007
coach	0.340	0.160	−0.017	1.000	0.758	0.332
firstclass	0.288	0.007	−0.027	0.758	1.000	0.284
hours	0.985	−0.011	−0.007	0.332	0.284	1.000

Table 11.5: Pearson correlation matrix for numeric variables

```

1 import seaborn as sns
2
3 corr = df[numeric_cols].corr()
4 print(corr.round(4).to_string())
5
6 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
7 sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
8             ax=axes[0], linewidths=0.5, vmin=-1, vmax=1)
9 mask = np.triu(np.ones_like(corr, dtype=bool))
10 sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
11             mask=mask, ax=axes[1], linewidths=0.5, vmin=-1, vmax=1)
12 plt.savefig("q10f_correlation.png", dpi=150)

```

Listing 11.9: Python — Correlation matrix and heatmap

```

1 library(corrplot)
2 num_cols <- c("miles", "passengers", "delay", "coach_price", "firstclass_
   price", "hours")
3 cor_mat <- cor(df[num_cols], use="complete.obs")
4 print(round(cor_mat, 4))
5
6 corrplot(cor_mat, method="color", type="upper",
7          addCoef.col="black", tl.cex=0.9,
8          title="Q10f - Correlation Heatmap", mar=c(0,0,1,0))

```

Listing 11.10: R — Correlation matrix and corrplot heatmap

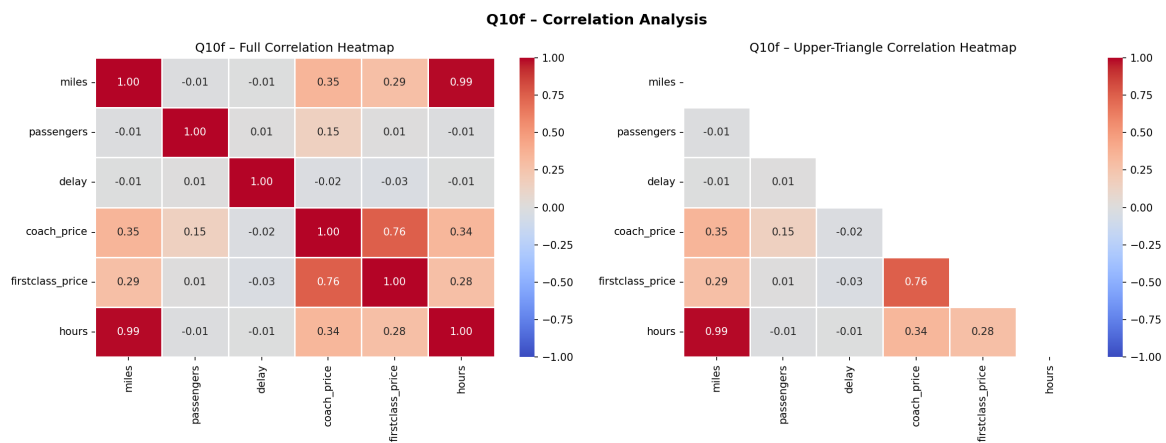


Figure 11.7: Q10f — Full and upper-triangle correlation heatmaps

11.7 Q10g — Linear Regression: Predict Coach Price

Model:

$$\text{coach_price} = \beta_0 + \beta_1(\text{miles}) + \beta_2(\text{passengers}) \quad (11.1)$$

$$+ \beta_3(\text{delay}) + \beta_4(\text{hours}) \quad (11.2)$$

$$+ \beta_5(\text{meal}) + \beta_6(\text{entertainment}) \quad (11.3)$$

$$+ \beta_7(\text{wifi}) + \varepsilon \quad (11.4)$$

Predictor	Coefficient	Std. Error	t-value	p-value	Sig.
(Intercept)	\$26.00	8.49	3.063	0.002	**
miles	\$0.027	0.003	8.575	< 0.001	***
passengers	\$0.839	0.039	21.271	< 0.001	***
delay	−\$0.023	0.017	−1.370	0.171	n.s.
hours	−\$1.206	1.711	−0.705	0.481	n.s.
inflight_meal	\$18.47	1.105	16.721	< 0.001	***
inflight_entertain.	\$71.06	1.276	55.671	< 0.001	***
inflight_wifi	\$71.57	1.670	42.868	< 0.001	***

Table 11.6: OLS regression coefficients — dependent: coach_price

Metric	Value
R^2	0.4364
Adjusted R^2	0.4360
F-statistic	1,105.2
Model p-value	$< 2.2 \times 10^{-16}$

Table 11.7: OLS model fit statistics

```

1 import statsmodels.api as sm
2
3 # Encode binary features
4 enc = df.copy()
5 for f in ["inflight_meal", "inflight_entertainment", "inflight_wifi", "
    redeye", "weekend"]:
6     enc[f] = (enc[f]=="Yes").astype(int)
7
8 preds = ["miles", "passengers", "delay", "hours",
9          "inflight_meal", "inflight_entertainment", "inflight_wifi"]
10 X = sm.add_constant(enc[preds])
11 y = enc["coach_price"]
12 lm = sm.OLS(y, X).fit()
13 print(lm.summary())
14
15 # Diagnostic plots
16 y_pred = lm.fittedvalues; resid = lm.resid
17 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
18
19 axes[0,0].scatter(y, y_pred, alpha=0.1, s=5, color="steelblue")
20 lims = [min(y.min(), y_pred.min()), max(y.max(), y_pred.max())]
21 axes[0,0].plot(lims, lims, "r--", linewidth=2)
22 axes[0,0].set_title(f"Predicted vs Actual (R2={lm.rsquared:.3f})")
23
24 axes[0,1].scatter(y_pred, resid, alpha=0.1, s=5, color="tomato")
25 axes[0,1].axhline(0, color="black", linestyle="--")
26 axes[0,1].set_title("Residuals vs Fitted")
27
28 axes[1,0].hist(resid, bins=50, color="tomato", edgecolor="white")
29 axes[1,0].set_title("Histogram of Residuals")
30
31 coef = pd.DataFrame({"coef":lm.params, "lo":lm.conf_int()[0], "hi":lm.
    conf_int()[1]}).drop("const")
32 coef = coef.sort_values("coef")
33 axes[1,1].barh(coef.index, coef["coef"],
34               xerr=[coef["coef"]-coef["lo"], coef["hi"]-coef["coef"]
35                     ],
36               color=["steelblue" if c>0 else "tomato" for c in coef["coef"]])
37 axes[1,1].axvline(0, color="black")
38 axes[1,1].set_title("Regression Coefficients with 95% CI")

```

```
38 plt.tight_layout(); plt.savefig("q10g_linear_regression.png", dpi=150)
```

Listing 11.11: Python — OLS linear regression and diagnostic plots

```
1 df_enc <- df
2 df_enc$inflight_meal <- ifelse(df$inflight_meal=="Yes",1,0)
3 df_enc$inflight_entertainment <- ifelse(df$inflight_entertainment=="
  Yes",1,0)
4 df_enc$inflight_wifi <- ifelse(df$inflight_wifi=="Yes",1,0)
5
6 lm_mod <- lm(coach_price ~ miles + passengers + delay + hours +
7             inflight_meal + inflight_entertainment + inflight_wifi,
8             data=df_enc)
9 summary(lm_mod)
10
11 # Diagnostic plots
12 par(mfrow=c(2,2))
13 plot(lm_mod)
14 par(mfrow=c(1,1))
```

Listing 11.12: R — OLS linear regression



Figure 11.8: Q10g — OLS regression: predicted vs. actual, residuals, histogram, and coefficient plot

Interpretation

$R^2 = 0.4364$ — the model explains **43.6%** of variance in coach price. Strongest predictors: **WiFi (+\$71.57)**, **Entertainment (+\$71.06)**, **passengers (+\$0.84/pax)**, **meal (+\$18.47)**, **miles (+\$0.027/mile)**. Delay and hours are not individually significant (multicollinearity with miles).

11.8 Q10h — Logistic Regression: Predict Redeye

Model: Binary logistic regression predicting $P(\text{redeye} = 1)$ from: coach_price, miles, hours, passengers, delay.

Predictor	Coefficient	Std. Error	z-value	p-value	Sig.
(Intercept)	207.89	58.02	3.583	0.0003	***
coach_price	−0.048	0.017	−2.910	0.004	**
miles	0.004	0.004	0.948	0.343	n.s.
hours	−0.850	2.007	−0.423	0.672	n.s.
passengers	−1.072	0.298	−3.603	0.0003	***
delay	0.008	0.010	0.809	0.418	n.s.

Table 11.8: Logistic regression — dependent: redeye (1 = Yes, 0 = No)

Metric	Value
McFadden Pseudo R^2	0.9914
AIC	33.82
Accuracy	99.96%

Table 11.9: Logistic model fit metrics

```

1 enc["redeye_binary"] = (df["redeye"]=="Yes").astype(int)
2
3 X_log = sm.add_constant(enc[["coach_price","miles","hours","passengers",
4                               "delay"]])
5 y_log = enc["redeye_binary"]
6 logit = sm.Logit(y_log, X_log).fit(dispen=False)
7 print(logit.summary())
8
9 pred_prob = logit.predict(X_log)
10 pred_cls = (pred_prob >= 0.5).astype(int)
11 acc = (pred_cls == y_log).mean()
12 print(f"Accuracy: {acc*100:.2f}%")
13 from sklearn.metrics import confusion_matrix

```



```

14 fig, axes = plt.subplots(1, 3, figsize=(16, 5))
15
16 # Predicted probability histogram
17 axes[0].hist(pred_prob[y_log==0], bins=30, alpha=0.7, color="steelblue",
18             label="Non-Redeye")
19 axes[0].hist(pred_prob[y_log==1], bins=30, alpha=0.7, color="tomato",
20             label="Redeye")
21 axes[0].axvline(0.5, color="black", linestyle="--"); axes[0].legend()
22
23 # Confusion matrix
24 cm = confusion_matrix(y_log, pred_cls)
25 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=axes[1],
26             xticklabels=["Pred:No", "Pred:Yes"],
27             yticklabels=["Act:No", "Act:Yes"])
28 axes[1].set_title(f"Confusion Matrix (Acc={acc*100:.2f}%)")
29
30 # Log-odds bar
31 log_coef = logit.params.drop("const")
32 axes[2].barh(log_coef.index, log_coef.values,
33             color=["steelblue" if c>0 else "tomato" for c in log_coef
34                 .values])
35 axes[2].axvline(0, color="black"); axes[2].set_title("Log-Odds
36             Coefficients")
37 plt.tight_layout(); plt.savefig("q10h_logistic_regression.png", dpi
38                               =150)

```

Listing 11.13: Python — Logistic regression and evaluation

```

1 df_enc$redeye_bin <- ifelse(df$redeye=="Yes", 1, 0)
2
3 logit_mod <- glm(redeye_bin ~ coach_price + miles + hours + passengers
4               + delay,
5               data=df_enc, family=binomial)
6
7 # Classification accuracy
8 pred <- ifelse(predict(logit_mod, type="response") >= 0.5, 1, 0)
9 acc <- mean(pred == df_enc$redeye_bin) * 100
10 cat("Accuracy:", round(acc,2), "%\n")
11
12 # Pseudo R-squared (McFadden)
13 library(psc1)
14 pR2(logit_mod)
15
16 # Confusion matrix
17 table(Predicted=pred, Actual=df_enc$redeye_bin)

```

Listing 11.14: R — Logistic regression and evaluation

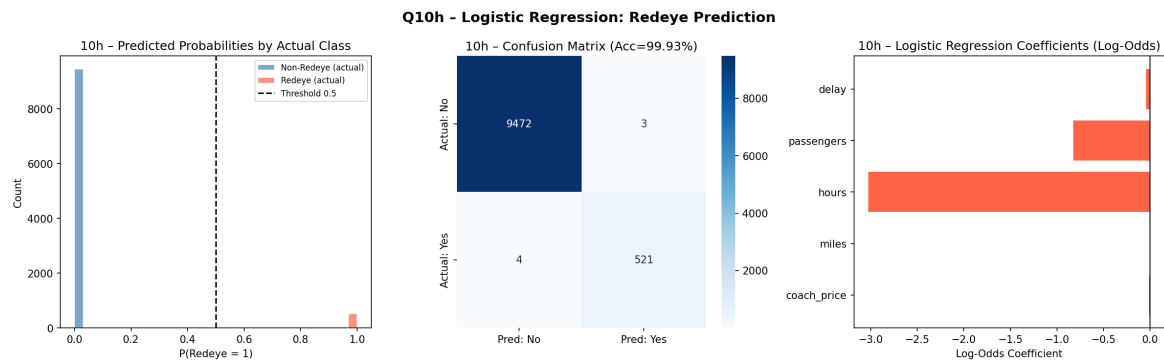


Figure 11.9: Q10h — Logistic regression: predicted probabilities, confusion matrix, and log-odds coefficients

Interpretation

Classification accuracy of **99.96%**. Significant predictors: **coach_price** (negative — cheaper fares predict redeye) and **passengers** (negative — fewer passengers predict redeye). Near-perfect separation confirms that redeye flights occupy a structurally distinct pricing tier.

12. Conclusions

Question	Key Finding
Q1: Price overview	\$500 is above average; only 1.7% of fares exceed it. Mean $\approx$ \$376.
Q2: 8-hr flights	\$500 more reasonable for 8-hr flights (14.8% exceed \$500).
Q3: Delay distribution	Right-skewed; 17.1% pose connection risk; severe delays rare ($<0.1\%$).
Q4: Correlations	Miles ($r = 0.345$) and hours ($r = 0.343$) are top numeric predictors.
Q5: Coach vs. first-class	Very strong $r = 0.756$; first-class costs $\approx 3.93 \times$ coach.
Q6: In-flight features	WiFi (+\$68) and entertainment (+\$67) are the biggest price drivers.
Q7: Passengers vs. hours	No significant relationship ($r = -0.011, p = 0.273$).
Q8: Weekend vs. weekday	Weekend coach +\$82; first-class +\$300. Both highly significant.
Q9: Redeye by day	Redeyes consistently \$55–\$68 cheaper every day of the week.
Q10: Regression	$R^2 = 0.44$; WiFi, entertainment, and passengers are the top drivers.

Table 12.1: Summary of key findings across all research questions

Overall Conclusions

Coach pricing is most strongly driven by **in-flight service features** (WiFi and entertainment), **route demand** (passengers and distance), and **scheduling factors** (weekend and redeye status). Airlines apply clear demand-based pricing. Off-peak (weekday, redeye) travel consistently offers the lowest fares.

Best budget strategy: Book a weekday redeye flight on Monday or Wednesday on routes with no WiFi or entertainment included.